RAGRouter-Bench: A Dataset and Benchmark for Adaptive RAG Routing

Ziqi Wang[1,†], **Xi Zhu**[1,†], **Shuhang Lin**[1], **Haochen Xue**[2], **Minghao Guo**[1] **Yongfeng Zhang**[1*]
[1]Rutgers University [2]University of Liverpool

{ziqi.wang0908, xi.zhu, shuhang.lin, minghao.guo, yongfeng.zhang}@rutgers.edu
haochen@liverpool.ac.uk

Code: https://github.com/ziqiwang0908/RAGRouter-Bench
Dataset: https://huggingface.co/datasets/Chaplain0908/RAGRouter
Leaderboard: https://huggingface.co/spaces/Chaplain0908/RAGRouter-Leaderboard

Abstract

Retrieval-augmented generation (RAG) has evolved into a family of paradigms with distinct performance profiles and resource demands, making paradigm selection a context-dependent, multi-criteria decision. Yet existing studies largely focus on isolated method improvements or query-only evaluation, without systematically examining RAG paradigms across diverse query–corpus contexts and effectiveness–efficiency trade-offs. We introduce RAGRouter-Bench, the first dataset and benchmark for adaptive RAG routing. It evaluates representative RAG paradigms under a unified retriever and models query–corpus compatibility by pairing three canonical query types with fine-grained structural and semantic corpus indicators. Each query, corpus, and paradigm combination is evaluated for both generation quality and token cost using quantitative metrics and LLM-as-a-Judge across two backbone LLMs. We further formulate routing as context-dependent paradigm selection and benchmark traditional and neural routers using query-only, corpus-only, and joint features. Our results show that no paradigm is universally optimal, neither query nor corpus signals alone are sufficient, and joint routing achieves effectiveness–efficiency trade-offs unattainable by any fixed paradigm. The remaining gap to the oracle reveals substantial unexploited query–corpus signal, establishing query–corpus compatibility as a central principle for adaptive RAG and RAGRouter-Bench as a systematic testbed for future routing methods. Our dataset and code are available at https://anonymous.4open.science/r/RAGRouter_Bench-1650/.

1 Introduction

While LLMs are prone to hallucinations in specialized or evolving domains (Ji et al., 2023; Mallen et al., 2023), retrieval-augmented generation (RAG) provides factual and faithful grounding through a two-stage "retrieve-then-generate" pipeline (Lewis et al., 2020; Guu et al., 2020; Gao et al., 2023). Retrieval is the primary bottleneck in RAG (Jin et al., 2024): it both defines the information boundary (Cao et al., 2024) and dominates computational overhead, making paradigm selection critical.

Existing RAG paradigms represent an evolution of retrieval strategies (Gao et al., 2023). NaiveRAG uses similarity-based retrieval for efficient factoid QA and summarization (Karpukhin et al., 2020), while GraphRAG adopts graph retrieval for multi-hop reasoning (Edge et al., 2024; Jiménez Gutiérrez et al., 2024; He et al., 2024; Sun et al., 2024). HybridRAG fuses vector and graph retrieval (Sarmah et al., 2024), whereas IterativeRAG invokes retrieval iteratively, trading efficiency for reasoning (Asai et al., 2024). Their distinct strengths and resource demands make paradigm choice a multi-criteria decision, motivating adaptive RAG routing (Jeong et al., 2024; Tang et al., 2025).

Preliminary experiments comparing representative paradigms across corpora (Figure 1) yield three insights. (i) No single paradigm consistently dominates across settings, indicating the absence of a one-size-fits-all solution; (ii) the optimal paradigm depends not only on query characteristics but also critically on the underlying corpus; (iii) greater so-

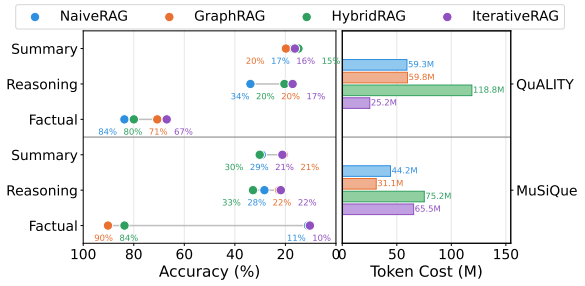


Figure 1: **Paradigm conflict in accuracy and token consumption across four RAG paradigms.**

*Corresponding author.
†These authors contributed equally to this work.

Table 1: **Comparison with existing RAG benchmarks.** Query and Corpus columns indicate whether the benchmark provides multi-type query coverage and corpus-level analysis. Routing indicates whether the benchmark supports adaptive paradigm selection.

Benchmark	Design			Query Type			Corpus Analysis		Evaluation	
	Query	Corpus	Routing	Factual	Reasoning	Summary	Semantic	Structural	Effectiveness	Efficiency
MultiHop-RAG (Tang and Yang, 2024)	✓	✗	Fixed	✗	✓	✗	✗	✗	✓	✗
WebQSP (Yih et al., 2016)	✓	✗	Fixed	✓	✓	✗	✗	✗	✓	✓
GraphRAG-Bench (Xiang et al., 2025)	✓	✗	Fixed	✓	✓	✓	✗	✓	✓	✗
GraphRAG-Bench (Xiao et al., 2025)	✓	✗	Fixed	✓	✓	✓	✗	✓	✓	✓
MemoRAG (Qian et al., 2024)	✗	✓	Fixed	✗	✓	✗	✗	✗	✓	✓
RAGRouter-Bench (Ours)	✓	✓	**Adaptive**	✓	✓	✓	✓	✓	✓	✓

phistication does not guarantee better performance, as simpler alternatives can achieve comparable results with lower overhead. These findings show adaptive RAG routing depends on query–corpus compatibility and efficiency trade-offs, requiring systematic benchmarking across queries, corpora, and RAG paradigms.

Nevertheless, existing datasets and benchmarks fall short in three ways (Table 1). First, they attribute optimal RAG selection solely to query complexity, overlooking the semantic and structural properties of the corpus and, more fundamentally, the role of query–corpus compatibility (Jeong et al., 2024; Tang et al., 2025). Second, they lack fine-grained, quantifiable indicators on either side, hindering accurate and interpretable decisions (Gao et al., 2023; Peng et al., 2024). Third, real-world RAG implementations are highly heterogeneous without standardized design protocols, making fair comparison inherently difficult. These gaps preclude systematic evaluation across query–corpus combinations and ultimately impede the development of adaptive RAG systems (Chen et al., 2024a; Lyu et al., 2024; Friel et al., 2024; Jin et al., 2024).

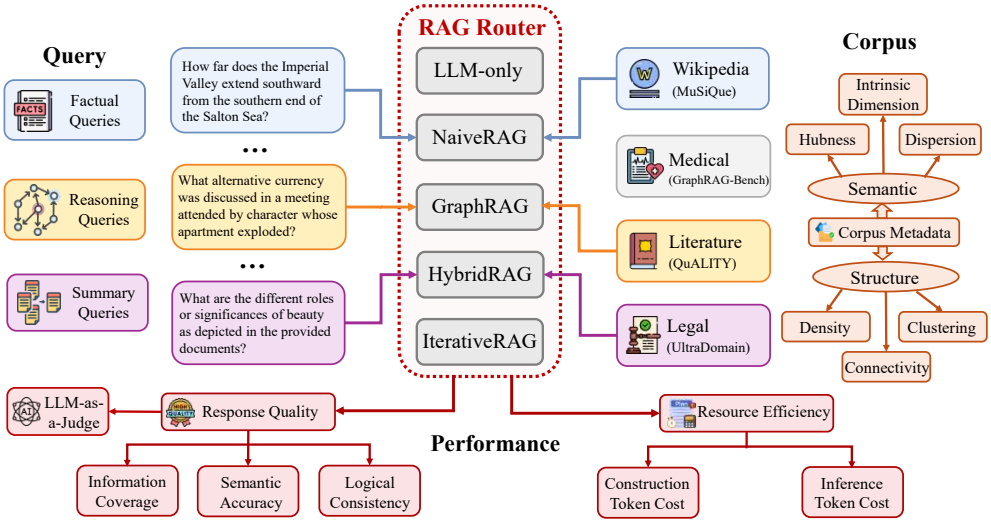
To this end, we introduce **RAGRouter-Bench**, the first dataset and benchmark for adaptive RAG routing. Grounded in query–corpus compatibility, it provides fine-grained, quantifiable indicators from both query and corpus perspectives, together with effectiveness and efficiency measurements across representative RAG paradigms. Each $(\text{query}, \text{corpus}, \text{paradigm}, \text{performance})$ tuple records one query–corpus combination evaluated under one paradigm. We further benchmark a suite of query–corpus routers on it, confirming the absence of a one-size-fits-all solution and the role of query–corpus compatibility. Our results show that adaptive routing achieves improved effectiveness–efficiency trade-offs, offering actionable insights toward next-generation RAG systems. Our contributions are as follows:

- **Standardized RAG Abstraction.** We formulate RAG paradigms under a unified retriever abstraction, enabling fair comparison across real-world RAG applications.
- **Dual-View Compatibility Framework.** We propose a hierarchical framework to characterize query–corpus compatibility, where three curated and augmented canonical query types are paired with multi-dimensional indicators of semantic and structural properties, revealing how query and corpus shape routing individually and interactively.
- **Effectiveness–Efficiency Protocol.** We establish a unified protocol pairing quantitative and LLM-as-a-judge quality metrics with construction and inference token costs.
- **Systematic Benchmarking.** We implement five standardized paradigms (LLM-only, NaiveRAG, GraphRAG, HybridRAG, IterativeRAG) over two backbones and four corpora, characterizing effectiveness–efficiency trade-offs across query–corpus pairs.
- **Validation and Insights.** We benchmark a range of query–corpus routers on RAGRouter-Bench, validating query–corpus compatibility and its potential for adaptive RAG systems.

2 Related Work

RAG Benchmarks and Evaluation. Current benchmarks primarily target specific retrieval challenges such as multi-hop reasoning (Tang and Yang, 2024; Yang et al., 2018), entity-centric QA (Yih et al., 2016), graph-augmented retrieval (Xiang et al., 2025; Xiao et al., 2025), and long-document understanding (Qian et al., 2024). Automated evaluation frameworks (Es et al., 2024; Saad-Falcon et al., 2024; Pradeep et al., 2025; Liu et al., 2025) and unified toolkits (Jin et al., 2024) enable scalable paradigm comparison, while recent work introduces corpus-level diagnostics (Facco et al., 2017; Radovanović et al., 2010) and cost-aware evaluation (Laitenberger et al., 2025; Wu et al., 2026). All of them, however, evaluate isolated paradigms under fixed routing, leaving no systematic platform for benchmarking adaptive RAG routing under effectiveness–efficiency trade-offs.

Figure 2: **Overview of the RAGRouter-Bench framework. Left:** Query types with examples. **Center:** Five RAG paradigms as routing targets. **Right:** Multi-domain corpora with structural and semantic profiles. **Bottom:** Dual-axis evaluation of response quality and resource efficiency.



Adaptive RAG Routing. Adaptive RAG strategies fall into three categories, namely query-complexity-driven paradigm selection (Jeong et al., 2024; Yang et al., 2026), selective retrieval triggering based on model confidence or external signals (Jiang et al., 2023; Su et al., 2024; Yao et al., 2024; Cheng et al., 2024; Marina et al., 2025; Feng et al., 2026), and multi-source routing across retrieval strategies or knowledge types (Wu et al., 2025; Bai et al., 2025; Li et al., 2025; Zhang et al., 2025), often under cost-aware objectives (Wang et al., 2024; Su et al., 2025; Mishra, 2026). All three rely on query-side or generation-side signals alone, without modeling how corpus structure and semantics shape paradigm effectiveness, which is the gap our query-corpus compatibility framework addresses.

3 The RAGRouter-Bench

We construct RAGROUTER-BENCH to investigate how query and corpus characteristics jointly determine RAG paradigm selection (Figure 2). Formally, given a query–corpus pair $(q, \mathcal{C})$ and a candidate paradigm space Π, we seek $\pi^* = \arg\max_{\pi \in \Pi} \mathcal{U}(\pi; q, \mathcal{C})$, where the utility $\mathcal{U}$ captures task-specific objectives such as response quality, token consumption, or their trade-off. This formulation frames RAG routing as context-dependent paradigm selection, positioning query–corpus compatibility as its central determinant.

3.1 RAG Paradigm Instantiation

To enable principled cross-paradigm comparison, we decompose real-world RAG pipelines into two retriever primitives and instantiate five representative paradigms from them, as illustrated in Figure 12 of Appendix B.2.

Base Retrievers. We define two primitives: (i) NaiveRetriever, which performs dense vector retrieval by encoding queries and chunks and selecting top-K segments (Karpukhin et al., 2020). (ii) GraphRetriever, which operates over knowledge graphs by extracting seed entities, propagating relevance via Personalized PageRank (PPR), and retrieving text from high-scoring nodes.

RAG Paradigm Instances. Building on these retrievers, we define five paradigms to accommodate a broad spectrum of RAG designs: (i) LLM-only, a retrieval-free baseline that directly prompts the LLM (Petroni et al., 2019); (ii) NaiveRAG, which invokes NaiveRetriever once and concatenates retrieved chunks as context (Lewis et al., 2020); (iii) GraphRAG, which applies GraphRetriever to extract relevant triplets and enhance generation (Edge et al., 2024); (iv) HybridRAG, which merges both retrievers via Reciprocal Rank Fusion (Cormack et al., 2009; Sarmah et al., 2024); (v) IterativeRAG, which adopts a retrieve–generate–evaluate loop for iterative query decomposition (Asai et al., 2024; Trivedi et al., 2023). This controlled instantiation removes confounds from implementation-specific variants. Details are provided in Appendix B.

3.2 Data Curation and Augmentation

Corpus Sourcing. We integrate datasets spanning four domains: encyclopedic knowledge from Wikipedia (MuSiQue (Trivedi et al., 2022)), literature (QuALITY (Pang et al., 2022)), legal documentation (UltraDomain_legal (Qian et al., 2024)), and medical textbooks (GraphRAGBench_medical (Xiao et al., 2025)), totaling 21,460 documents.

Query Generation. We incorporate three canonical query types across all corpora: (i) factual queries for single-segment entity retrieval; (ii) reasoning queries for multi-hop inference; and (iii) summary queries for multi-source information aggregation. Existing benchmarks, however, are highly skewed. MuSiQue consists exclusively of reasoning queries, while QuALITY contains over 90% factual queries (Trivedi et al., 2022; Pang et al., 2022). We therefore apply query data augmentation via LLM-based structure-aware expansion (Xiao et al., 2025), validated by a verify-then-filter protocol (Chen et al., 2024a) whose automated judgments reach 94% agreement with human verification on a stratified sample (N=200). The resulting dataset comprises 7,727 queries, including 4,086 reasoning (52.9%), 2,320 factual (30.0%), and 1,321 summary queries (17.1%), with all corpora covering all query types to enable controlled cross-type comparison (Figure 3). Details are in Appendix A.2.

3.3 Dual-View Analysis

RAG performance should be governed by query–corpus interactions rather than query complexity alone. With three canonical query types for coverage, we turn to the corpus, which defines the retrieval environment, and characterize it with two complementary families of indicators. (i) Structural topology metrics capture properties that govern graph-based retrieval: largest connected component ratio (LCC ratio) measures global connectivity, where fragmentation impedes multi-hop reasoning (Newman, 2010); average and maximum degree centrality reflect connectivity strength, where sparsity limits relational bridging (Sun et al., 2024); relation type diversity captures edge semantic richness for precise traversal; and clustering coefficient measures local cohesiveness that facilitates evidence aggregation (Watts and Strogatz, 1998). (ii) Semantic space metrics characterize properties that affect vector-based retrieval: intrinsic dimension measures effective dimensionality, where high values degrade distance-based similarity (Facco et al., 2017); dispersion quantifies embedding uniformity, where low values cause semantic crowding (Wang and Isola, 2020); and hubness captures retrieval bias, where high values skew results toward frequent but potentially irrelevant passages (Radovanović et al., 2010). Together, these indicators give the router a computable description of the retrieval environment. Detailed definitions are in Appendix A.3.

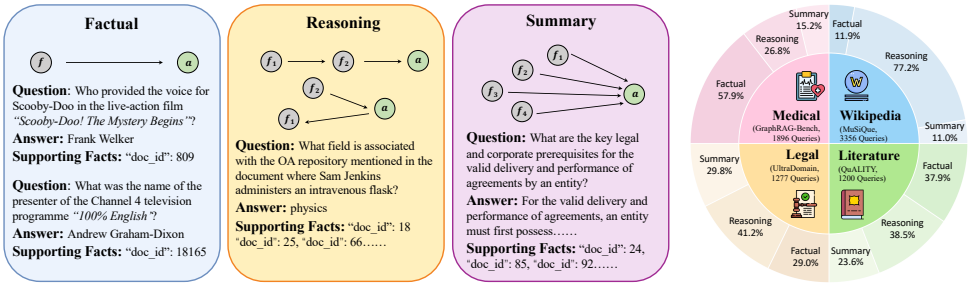
3.4 Evaluation Protocol for RAG Paradigms

We evaluate paradigms along two axes, effectiveness and efficiency (Jin et al., 2024). (i) Generation Quality. We use three complementary metrics: Semantic F1 for token-level accuracy (Zhang et al., 2020), coverage for key information completeness (Reimers and Gurevych, 2019), and faithfulness for factual grounding in retrieved content (Es et al., 2024). In addition, we use LLM-as-a-Judge for holistic correctness via ternary classification (Zheng et al., 2023). (ii) Resource Consumption. We decompose token usage into construction cost, the one-time preprocessing overhead such as knowledge graph construction amortized over queries, and inference cost, the per-query retrieval and generation overhead. Generation input is truncated to 8k tokens to respect LLM context limits. Metric definitions are in Appendix C.1, and the cost decomposition in Appendix C.3. Since judge outputs also serve as routing labels, we validate them along three axes: agreement with human annotation (Cohen's $\kappa = 0.803$), agreement with a second judge model from another vendor ($\kappa = 0.847$), and robustness to rubric phrasing, under which instance-level agreement stays high. Full protocols are in Appendix C.2.

3.5 Query-Corpus Router Design

To validate the practical operability of the proposed query–corpus compatibility, we formalize paradigm routing as a multi-class classification task over the established dataset. Given a query–corpus pair, the router encodes the complementary query-side features and corpus indicators of Section 3.3, projects them into a shared space for joint modeling of query–corpus interactions, and predicts the optimal RAG paradigm. Following Section 3.4, the optimal paradigm for each query–corpus pair is determined by a utility-driven mapping. Our main experiments adopt a quality-first label: among the paradigms judged correct by LLM-as-a-Judge, we assign the one attaining the highest Semantic F1. When no paradigm yields a satisfactory response, the pair is assigned a dedicated "*cannot answer*" label, which allows the router to recognize inherent task boundaries rather than forcing a suboptimal routing decision. For correct-rate accounting, such a pair is neither counted as correct nor as incorrect: it falls back to the most frequent non-abstaining paradigm of the training fold, and its correctness

Figure 3: **Query type taxonomy and dataset composition in RAGRouter-Bench. Left:** The three query types. **Right:** Their distribution across four datasets.



is read off that paradigm's generated answer. The same formulation admits cost-aware objectives by assigning $\arg\max_\pi \left(\mathrm{F1}_\pi - \lambda \widetilde{c}_\pi\right)$, where $\widetilde{c}_\pi$ is the normalized token cost of paradigm π and λ controls the quality–cost trade-off. Setting $\lambda = 0$ recovers the quality-first label. Implementation details on feature dimensions, encoders, and training procedures are in Section 4.1 and Appendix D.

4 Experiments

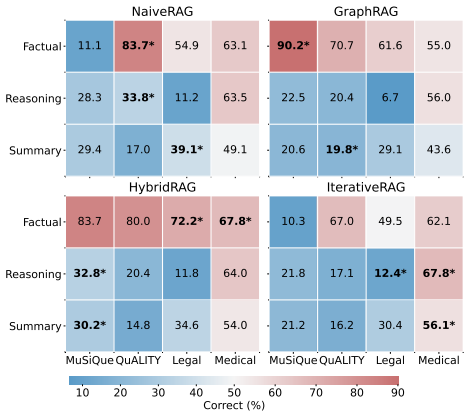
4.1 Experimental Setup

RAG Paradigms. All paradigms share the same generators, embedding model and context budget. We use DeepSeek-V3 (DeepSeek-AI, 2024) and LLaMA-3.1-8B (Team, 2024) as generators and text-embedding-3-small for vectorization (OpenAI, 2024), under an 8k token budget. The two backbones bracket the deployment spectrum RAG routing targets: DeepSeek-V3 is a 671B mixture-of-experts model behind a hosted API, whereas LLaMA-3.1-8B is an open-weight model running locally under budget or privacy constraints. Retrieval hyperparameters follow each paradigm's published protocol and are listed in Appendix B.1.

Evaluation. We compute the corpus indicators of Section 3.3 and evaluate all settings under the protocol of Section 3.4. For generation quality, we measure semantic F1 (Zhang et al., 2020), coverage, faithfulness (Es et al., 2024), and LLM-as-a-Judge accuracy with GPT-4o (OpenAI, 2023). For resource efficiency, we track token consumption split into retrieval and generation costs (Jin et al., 2024). These form the $(\text{query}, \text{corpus}, \text{paradigm}, \text{performance})$ tuples of RAGRouter-Bench. Metric implementations and cost decomposition are in Appendices C.1 and C.3.

Router Models. We evaluate two families of routers. Traditional ML routers are XGBoost (Chen and Guestrin, 2016) and MLP (Rumelhart et al., 1986), with logistic regression and SVM reported in Appendix D.4; neural routers are DNN, TwoTower, Adaptive-RAG (Jeong et al., 2024),

Figure 4: **Paradigm performance across datasets and query types.** Each panel gives one paradigm's LLM-as-a-Judge accuracy (%), with rows as query types and columns as datasets. An asterisk marks the best paradigm for each combination.



MBA-RAG (Tang et al., 2025), and RouterDC (Chen et al., 2024b), with query text encoded by BERT (Devlin et al., 2019) or BGE (Xiao et al., 2024). Feature processing and the training protocol are in Appendix D.2. Non-adaptive baselines bound the achievable range: Random samples uniformly; Best-fixed always selects the globally most frequent optimal paradigm; Always-π applies a single paradigm to every query, which upper-bounds any non-adaptive strategy in correct rate; and Oracle selects the ground-truth optimal paradigm per query. Reporting the full Always-π family matters because the most frequently optimal paradigm and the paradigm with the highest aggregate correct rate need not coincide. Routers are assessed by routing accuracy, macro F1, and correct rate, defined in Appendix D.3.

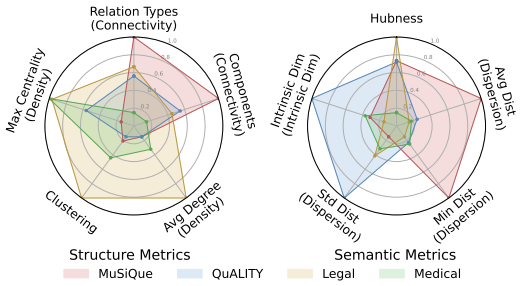
4.2 Results and Analysis

Comparative Paradigm Analysis. Table 2 and Figure 4 yield three insights. (i) No paradigm is universally optimal across corpora. GraphRAG achieves 90.2% on MuSiQue factual queries but drops to 70.7% on QuALITY, while NaiveRAG excels on the same query type in QuALITY (83.7%) but fails on MuSiQue (11.1%). These reversals

Table 2: **Main evaluation results across RAG paradigms, datasets, and backbone LLMs.** Each paradigm reports LLM-as-a-Judge accuracy (%) by query type and overall, with average token consumption per query. Green marks the best value and Blue the second best, ties sharing a color. Overall averages are weighted by the query-type counts of each corpus. The retrieval-free LLM-only baseline scores 0.0% on every dataset under both backbones and is omitted here (Table 18), and IterativeRAG denotes the NaiveRetriever-based variant (Appendix B.2).

Method	MuSiQue					QuALITY					Legal					Medical				
	Fac.	Rea.	Sum.	**Avg.**	Tok.	Fac.	Rea.	Sum.	**Avg.**	Tok.	Fac.	Rea.	Sum.	**Avg.**	Tok.	Fac.	Rea.	Sum.	**Avg.**	Tok.
Model: DeepSeek-V3																				
NaiveRAG	11.1	**28.3**	**29.4**	26.4	13k	**83.7**	**33.8**	**17.0**	**48.7**	50k	54.9	11.2	**39.1**	**32.2**	46k	**63.1**	63.5	49.1	61.1	51k
GraphRAG	**90.2**	22.5	20.6	**30.3**	9k	70.7	**20.4**	**19.8**	39.3	50k	**61.6**	6.7	29.1	29.3	184k	55.0	56.0	43.6	53.5	38k
HybridRAG	**83.7**	**32.8**	**30.2**	**38.6**	22k	**80.0**	**20.4**	14.8	**41.7**	99k	**72.2**	**11.8**	**34.6**	**36.1**	230k	**67.8**	**64.0**	**54.0**	**64.7**	74k
IterativeRAG	10.3	21.8	21.2	20.4	20k	67.0	17.1	16.2	35.8	21k	49.5	**12.4**	30.4	28.5	20k	62.1	**67.8**	**56.1**	**62.7**	7k
Model: LLaMA-3.1-8B																				
NaiveRAG	10.3	7.9	**12.0**	8.6	13k	**69.2**	10.9	**1.4**	**30.7**	50k	50.3	**10.7**	**22.8**	**25.8**	46k	**52.1**	37.9	30.1	**44.9**	51k
GraphRAG	**84.4**	**9.7**	11.4	**18.7**	9k	44.7	9.5	**2.5**	21.2	50k	**55.4**	7.4	19.7	25.0	184k	48.1	33.6	**31.8**	41.7	38k
HybridRAG	**79.9**	**12.1**	**13.9**	**20.3**	22k	**70.3**	**15.6**	**2.5**	**33.2**	99k	**63.8**	**13.1**	**24.2**	**31.1**	230k	**55.7**	**39.9**	**33.9**	**48.2**	74k
IterativeRAG	12.3	6.0	6.8	6.8	20k	62.3	**11.5**	**1.4**	28.4	21k	48.7	10.1	12.6	22.0	20k	47.5	**44.6**	27.3	43.6	7k

Figure 5: **Corpus features across structural and semantic dimensions. Left:** Graph topology metrics. **Right:** Embedding space metrics.



align with corpus structure rather than query type: MuSiQue offers by far the richest relation vocabulary (44,766 relation types, Table 11), which entity-centric graph traversal can exploit, while its fragmented sources (21,153 chunks drawn from documents averaging only 107.9 tokens) give dense retrieval little to match against; QuALITY inverts both properties, pairing a graph less than half the size with article-scale documents (5,741.1 tokens on average) that direct vector matching resolves easily. (ii) Paradigm effectiveness depends on query type. Within the same corpus, the optimal paradigm shifts with task complexity: on MuSiQue, from GraphRAG for factual (90.2%) to HybridRAG for reasoning (32.8%) and summarization (30.2%); in the Medical domain, IterativeRAG is strongest on reasoning queries whereas HybridRAG is strongest on factual ones, the two paradigms reaching an identical 67.8% on their respective query types. (iii) RAG paradigm selection is therefore a multi-criteria decision problem that no single paradigm can settle, which motivates adaptive routing grounded in both query characteristics and corpus properties. Query-level paired bootstrap resampling (10,000 iterations) confirms that this ordering is stable: the 95% confidence intervals of the four retrieval paradigms are mutually non-overlapping and the full ranking is reproduced in 99.95% of resamples (Appendix C).

Router Benchmarking Analysis. Table 3 reports all routers under joint query–corpus features. (i) Neural routers are better balanced. They achieve higher F1 and correct rate, while traditional ML routers stay competitive in routing accuracy but fall well behind in F1, indicating strong majority-class bias. Within the same architecture, BERT-based encodings enable more balanced paradigm selection than BGE embeddings. (ii) A stronger backbone makes routing harder but the outcome better. LLaMA-3.1-8B achieves higher routing accuracy than DeepSeek-V3 yet lower correct rate, because weaker generation concentrates labels toward "cannot answer", simplifying the classification task but limiting downstream correctness. Routing difficulty is thus intrinsically coupled with generation quality, rather than purely a modeling issue. (iii) Routing targets the effectiveness–efficiency frontier rather than accuracy alone. Table 4 compares every non-adaptive strategy against routing on the accuracy–cost plane. Always HybridRAG attains the highest correct rate among fixed paradigms (45.04%), but only at the largest token budget; the fixed paradigm operating at comparable cost to the router, Always NaiveRAG ($0.41\times$), reaches just 39.32%. Routing recovers 95% of HybridRAG's correct rate at $0.43\times$ its cost, an operating point no single paradigm occupies, and the same ordering holds on LLaMA-3.1-8B. The remaining gap to Oracle (60.83%) is the part of the query–corpus routing signal that current routers still leave unexploited, indicating that adaptive RAG routing remains a largely open problem.

Query-Corpus Compatibility Analysis. Figure 5

Table 3: **Router evaluation under query–corpus features.** R.A is routing accuracy, F1 is macro F1, and C.R is correct rate, all in percent. Green marks the best value in a column and Blue the second best, both computed over the learned routers only. Best-fixed applies NaiveRAG on DeepSeek-V3 and GraphRAG on LLaMA-3.1-8B. Class counts differ across backbones, which is why the Random baseline differs between the two panels.

Method	Avg			MuSiQue			QuALITY			Legal			Medical		
	R.A	F1	C.R	R.A	F1	C.R	R.A	F1	C.R	R.A	F1	C.R	R.A	F1	C.R
Model: DeepSeek-V3															
Random	20.06	18.90	39.06	20.04	–	28.40	19.96	–	42.84	20.05	–	31.61	20.16	–	60.58
Best-fixed	20.91	6.92	39.32	15.46	–	26.37	30.05	–	48.75	18.72	–	32.18	26.27	–	61.08
MLP	46.78	19.92	42.93	54.82	23.89	35.31	54.61	24.68	48.52	51.38	19.72	31.89	24.51	10.02	60.31
XGBoost	45.76	20.58	42.93	52.78	24.08	35.42	51.45	24.82	48.73	52.76	21.95	32.01	25.04	10.78	59.92
TwoTower (BGE)	43.29	25.41	42.28	49.71	29.02	32.33	45.97	23.48	48.30	52.49	25.58	33.19	24.02	20.12	62.22
TwoTower (BERT)	44.95	25.42	42.81	51.44	28.26	33.67	49.13	24.49	48.21	53.28	26.10	33.57	25.21	20.51	61.80
Adap-RAG (BGE)	43.93	26.24	42.43	49.98	29.55	33.22	47.55	25.04	48.88	51.79	25.10	32.64	25.65	21.92	61.24
Adap-RAG (BERT)	46.19	23.20	42.87	53.01	26.68	34.27	51.90	24.75	48.99	53.07	25.03	33.36	25.87	14.81	60.64
MBA-RAG (BGE)	43.30	25.04	41.61	48.48	26.67	30.73	47.65	23.59	48.72	50.06	27.51	32.98	26.82	21.40	62.17
MBA-RAG (BERT)	45.17	21.48	42.66	51.49	25.15	33.72	50.23	23.43	48.83	52.51	22.58	32.62	25.85	13.02	61.33
DNN (BGE)	43.92	29.76	43.37	50.95	35.15	35.01	44.77	24.91	46.92	51.03	27.51	32.99	26.15	24.78	62.90
DNN (BERT)	45.48	27.77	43.40	53.42	33.61	35.49	48.28	24.77	47.95	52.83	26.64	33.91	24.71	20.09	60.93
RouterDC (BGE)	43.84	28.82	42.84	50.20	34.56	34.18	44.77	23.80	47.08	52.91	27.49	32.70	25.87	22.72	62.32
RouterDC (BERT)	45.97	28.80	43.69	52.64	35.01	35.55	51.62	27.88	48.57	53.22	26.82	33.79	25.71	19.71	61.69
Oracle	100.00	100.00	60.83	100.00	–	51.61	100.00	–	59.93	100.00	–	50.51	100.00	–	84.65
Model: LLaMA-3.1-8B															
Random	24.95	21.87	26.57	24.92	–	16.62	25.18	–	26.58	24.94	–	26.65	24.86	–	44.13
Best-fixed	14.42	6.30	25.78	12.81	–	18.71	9.35	–	21.20	13.47	–	24.98	21.10	–	41.72
MLP	62.63	30.15	27.07	78.32	35.74	18.41	66.89	31.98	28.21	58.55	26.18	24.01	34.92	21.78	43.72
XGBoost	62.45	29.03	26.72	78.48	35.91	18.52	67.12	32.41	28.38	58.98	27.43	23.89	33.45	15.81	42.08
TwoTower (BGE)	59.91	29.91	27.06	73.23	30.72	18.61	62.83	35.15	26.22	61.28	32.51	25.10	33.57	23.43	43.88
TwoTower (BERT)	61.74	31.16	26.66	74.96	33.65	18.28	63.76	34.32	27.49	61.95	32.41	24.73	36.92	23.93	42.26
Adap-RAG (BGE)	60.25	30.40	26.88	73.56	31.32	18.67	62.53	31.43	26.06	61.11	34.23	25.14	34.69	25.56	43.11
Adap-RAG (BERT)	62.66	30.75	27.22	75.39	32.27	18.73	67.83	34.75	28.83	61.49	34.14	25.02	37.64	23.24	42.73
MBA-RAG (BGE)	59.18	29.16	27.14	72.76	29.74	18.50	62.48	31.81	27.24	58.48	32.31	25.66	33.53	24.32	43.37
MBA-RAG (BERT)	62.85	30.49	27.15	75.77	31.36	18.73	66.87	33.79	27.72	62.44	35.07	25.46	37.73	23.80	42.83
DNN (BGE)	58.33	34.61	27.40	72.14	37.50	18.70	60.43	35.41	27.56	58.35	35.26	25.09	32.53	28.54	44.27
DNN (BERT)	61.42	34.65	27.38	74.67	38.06	18.79	65.23	35.93	27.94	61.80	35.68	25.68	35.30	27.10	43.37
RouterDC (BGE)	60.01	33.81	27.09	73.98	38.20	18.92	60.49	32.91	26.65	59.49	34.97	25.62	35.32	25.81	42.83
RouterDC (BERT)	61.30	34.52	27.20	73.77	37.51	18.86	65.56	38.68	28.03	61.37	34.81	25.31	36.48	26.42	42.73
Oracle	100.00	100.00	40.57	100.00	–	26.76	100.00	–	39.73	100.00	–	41.19	100.00	–	65.14

shows the structural and semantic profile of each domain and Figure 6 the routing performance under each feature configuration. (i) Joint query–corpus features are necessary for effective routing. The joint configuration consistently outperforms query-only and corpus-only across all dataset–model settings in both routing accuracy and correct rate. On MuSiQue, routing accuracy improves from 48.4% to 55.3% and correct rate from 26.4% to 35.8%, demonstrating the complementary nature of query and corpus signals. (ii) Single-sided signals are fundamentally insufficient. Corpus-only features yield nearly identical performance across routers due to their dataset-level invariance, essentially collapsing routing into a fixed per-dataset mapping. While query-only features provide more granular signals, their effectiveness is inconsistent across diverse domains, further justifying the necessity of a unified dual-view representation. (iii) Corpus structure and semantics shape paradigm effectiveness. The Legal corpus exhibits the densest graph structure (average degree 3.86, LCC ratio 0.933, Table 11) alongside the most congested embedding space (hubness 1.46, centroid dispersion 0.300, Table 12), explaining why HybridRAG outperforms NaiveRAG on factual queries there (72.2% versus 54.9%). Conversely, QuALITY offers roughly half MuSiQue's relation vocabulary (23,828 types) over a graph less than half the size (90,088 nodes), limiting what graph traversal can recover, while its embedding space is less crowded than Legal's (dispersion 0.345), allowing NaiveRAG to dominate on factual queries (83.7%) via direct semantic matching. These fine-grained metrics provide the computable signals necessary for context-aware routing decisions.

Cost-Performance Trade-off Analysis. Figure 7 presents the cost–effectiveness landscape across RAG paradigms and the router. (i) No fixed paradigm is Pareto-optimal across datasets. While

Figure 6: **Routing performance under different input signals.** Average R.A and C.R (%) over all nine routers under query-only (Q), corpus-only (C), and joint (Q+C) features, bounded by Random and Oracle.

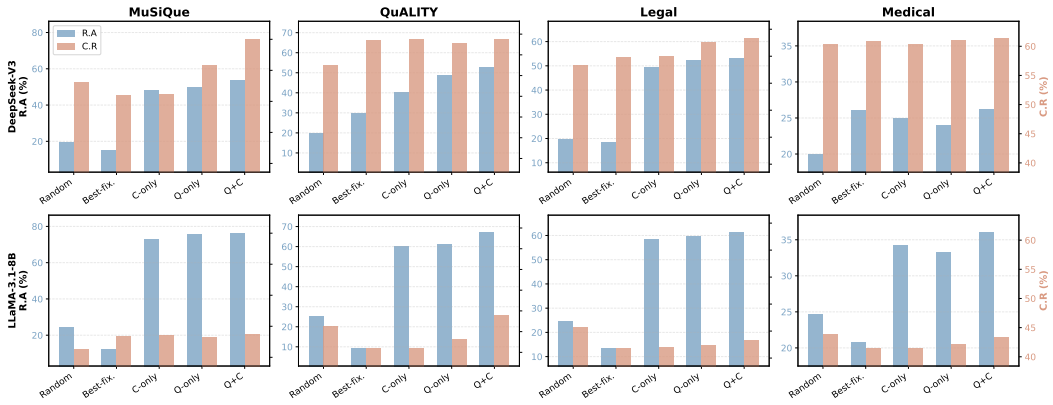
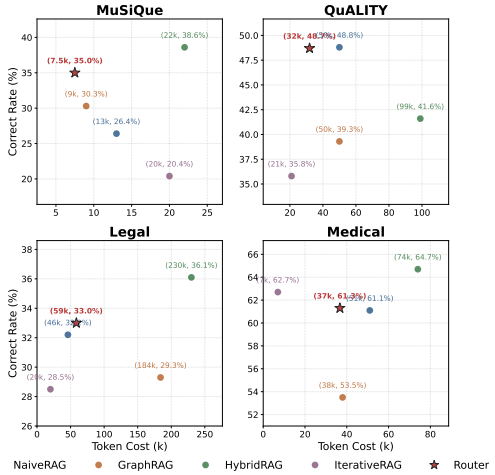


Table 4: **Accuracy–cost comparison of non-adaptive strategies and routing (DeepSeek-V3).** Cost is the total token consumption over the full 7,727-query benchmark, with ratios relative to Always HybridRAG, the most expensive strategy. C.R is correct rate (%).

Strategy	Cost (M)	Ratio	C.R
Always LLM-only	0.29	0.00	0.00
Always IterativeRAG	129.42	0.21	34.50
Always NaiveRAG	258.58	0.41	39.32
Always GraphRAG	397.13	0.63	37.23
Always HybridRAG	627.28	1.00	**45.04**
Router (XGBoost)	267.01	0.43	**42.93**
Oracle	–	–	60.83

HybridRAG delivers the highest correct rate on most datasets, it incurs a prohibitive computational burden (e.g., 230k tokens per query on Legal), whereas lower-cost paradigms reduce token usage but incur accuracy degradation. No single fixed strategy simultaneously occupies the upper-left corner across all four datasets, confirming that optimal RAG selection is a multi-criteria decision governed by specific deployment constraints. (ii) The router achieves near-optimal effectiveness at significantly lower cost. On MuSiQue it attains 35.0% correct rate at 7.5k tokens against HybridRAG's 38.6% at 22k, and on Legal 33.0% at 58.6k against 36.1% at 230k, cost reductions of 66% and 75%. Across datasets, router solutions consistently lie closer to the Pareto frontier, indicating improved cost–effectiveness trade-offs. These savings come from paradigm selection rather than from shifting cost elsewhere: the routers are non-generative classifiers that consume no tokens and add sub-millisecond prediction latency. Making cost part of the label itself compresses the frontier further, which we analyze in Appendix C.4. (iii) Token consumption spans over an order of magnitude across paradigms, so deployment constraints are a primary factor in selecting RAG strategies, which

Figure 7: **Cost-effectiveness trade-off across RAG paradigms and router (DeepSeek-V3).** Each point is one strategy on a dataset, with token cost per query on the x-axis and correct rate on the y-axis.



RAGRouter-Bench captures by jointly modeling effectiveness and efficiency.

5 Conclusion

We introduce RAGRouter-Bench, the first benchmark for adaptive RAG routing. It evaluates five RAG paradigms across four corpora and three query types under a unified effectiveness–efficiency protocol, characterizes corpus properties with computable structural and semantic indicators, and benchmarks traditional and neural routers. Results show that no paradigm is universally optimal, as performance depends jointly on query characteristics and corpus properties. Routing with both signals preserves most of the accuracy of the strongest fixed paradigm at much lower cost, framing RAG selection as a context-dependent decision rather than a static pipeline choice. The remaining gap to the oracle calls for better query–corpus representations and routing mechanisms. RAGRouter-Bench provides a foundation for adaptive, interpretable, and cost-aware RAG systems centered on

query–corpus compatibility.

6 Limitations

RAGRouter-Bench instantiates query–corpus routing over four corpus domains and five RAG paradigms; extending either axis reuses the pipeline described in Appendix E without changes to the benchmark design. All corpora are in English, so we do not characterize how retrieval environments differ across languages. Finally, we quantify efficiency in tokens, the unit that transfers across deployments, while the monetary cost and wall-clock latency they translate into depend on the serving stack and lie outside our scope.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

Haoyue Bai, Haoyu Wang, Shengyu Chen, Zhengzhang Chen, Lu-An Tang, Wei Cheng, Haifeng Chen, and Yanjie Fu. 2025. Learning to route: A rule-driven agent framework for hybrid-source retrieval-augmented generation. *arXiv preprint arXiv:2510.02388*.

Bowen Cao, Deng Cai, Leyang Cui, Xuxin Cheng, Wei Bi, Yuexian Zou, and Shuming Shi. 2024. Retrieval is accurate generation. In *The Twelfth International Conference on Learning Representations*.

Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024a. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17754–17762.

Shuhao Chen, Weisen Jiang, Baijiong Li, Pengcheng Shi, and Caihua Zhang. 2024b. Routerdc: Query-based router by dual contrastive learning for assembling large language models. *arXiv preprint arXiv:2409.19886*.

Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Qinyuan Cheng, Xiaonan Li, Shimin Li, Qin Zhu, Zhangyue Yin, Yunfan Shao, Linyang Li, Tianxiang Sun, Hang Yan, and Xipeng Qiu. 2024. Unified active retrieval for retrieval augmented generation. In *Findings of EMNLP*.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 411–418.

DeepSeek-AI. 2024. DeepSeek-V3 technical report. *Preprint*, arXiv:2412.19437.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL*, pages 4171–4186.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*, pages 1383–1392.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024)*.

Elena Facco, Maria d'Errico, Alex Rodriguez, and Alessandro Laio. 2017. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7(1):1–8.

Jun Feng, Jiahui Tang, Zhicheng He, Hang Lv, Hongchao Gu, Hao Wang, Xuezhi Yang, and Shuai Fang. 2026. Rethinking the necessity of adaptive retrieval-augmented generation through the lens of adaptive listwise ranking. *arXiv preprint arXiv:2604.15621*.

Robert Friel, Masha Belyi, and Atindriyo Sanyal. 2024. RAGBench: Explainable benchmark for retrieval-augmented generation systems. *arXiv preprint arXiv:2407.11005*.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, pages 3929–3938.

Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson,

and Bryan Hooi. 2024. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. *arXiv preprint arXiv:2402.07630*.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2024)*, pages 1–15.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of EMNLP*.

Bernal Jiménez Gutiérrez, Yu Gu, Yiheng Shu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Jiajie Jin, Yutao Zhu, Guanting Dong, Yuyao Zhang, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, Zhicheng Dou, and Ji-Rong Wen. 2024. FlashRAG: A modular toolkit for efficient retrieval-augmented generation research. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP 2004)*, pages 388–395.

Alex Laitenberger, Christopher D. Manning, and Nelson F. Liu. 2025. Stronger baselines for retrieval-augmented generation with long-context language models. In *Proceedings of EMNLP*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Kuan Li, Liwen Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Shuai Wang, and Minhao Cheng. 2025. Lara: Benchmarking retrieval-augmented generation and long-context llms – no silver bullet for lc or rag routing. In *Proceedings of ICML*.

Shuliang Liu, Xinze Li, Zhenghao Liu, Yukun Yan, Cheng Yang, Zheni Zeng, Zhiyuan Liu, Maosong Sun, and Ge Yu. 2025. Judge as a judge: Improving the evaluation of retrieval-augmented generation through the judge-consistency of large language models. *arXiv preprint arXiv:2502.18817*.

Yuanjie Lyu, Zhiyi Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Peng Wu, Ke Liu, Min Chen, and Xing Wan. 2024. CRUD-RAG: A comprehensive chinese benchmark for retrieval-augmented generation of large language models. *arXiv preprint arXiv:2401.17043*.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822.

Maria Marina, Nikolay Ivanov, Sergey Pletenev, Mikhail Salnikov, Daria Galimzianova, Nikita Krayko, Vasily Konovalov, Alexander Panchenko, and Viktor Moskvoretskii. 2025. Llm-independent adaptive rag: Let the question speak for itself. In *Proceedings of EMNLP*.

Sanjay Mishra. 2026. Cost-aware query routing in RAG: Empirical analysis of retrieval depth tradeoffs. *arXiv preprint arXiv:2606.02581*.

Mark Newman. 2010. *Networks: An Introduction*. Oxford University Press.

OpenAI. 2023. GPT-4 technical report. *Preprint*, arXiv:2303.08774.

OpenAI. 2024. New embedding models and API updates. `https://openai.com/blog/new-embedding-models-and-api-updates`.

Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Arpit Phan, Jason andoba, Phu Mon Htut, Jheng-Hong Xie, Samuel R. Bowman, and He He. 2022. QuALITY: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5336–5358.

Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*.

Fabio Petroni, Tim Rocktäschel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, Alexander H. Miller, and

Sebastian Riedel. 2019. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473.

Ronak Pradeep, Nandan Thakur, Shivani Upadhyay, Daniel Campos, Nick Craswell, and Jimmy Lin. 2025. The great nugget recall: Automating fact extraction and RAG evaluation with large language models. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2025)*.

Hongjin Qian, Peitian Zhang, Zheng Liu, Kelong Mao, and Zhicheng Dou. 2024. MemoRAG: Moving towards next-gen RAG via memory-inspired knowledge discovery. *arXiv preprint arXiv:2409.05591*.

Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.

David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*.

Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. 2024. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. *arXiv preprint arXiv:2408.04948*.

Jinyan Su, Jennifer Healey, Preslav Nakov, and Claire Cardie. 2025. Fast or better? balancing accuracy and cost in retrieval-augmented generation with flexible user control. *arXiv preprint arXiv:2502.12145*.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024. Dragin: Dynamic retrieval augmented generation based on the information needs of large language models. In *Proceedings of ACL*.

Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Xiaqiang Tang, Qiang Gao, Jian Li, Nan Du, Qi Li, and Sihong Xie. 2025. MBA-RAG: a bandit approach for adaptive retrieval-augmented generation through question complexity. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, pages 3248–3254.

Yixuan Tang and Yi Yang. 2024. MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, pages 2369–2380.

Llama Team. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, pages 10014–10029.

Tongzhou Wang and Phillip Isola. 2020. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *International Conference on Machine Learning (ICML)*, pages 9929–9939.

Ziting Wang, Haitao Yuan, Wei Dong, Gao Cong, and Feifei Li. 2024. Corag: A cost-constrained retrieval optimization system for retrieval-augmented generation. *arXiv preprint arXiv:2411.00744*.

Duncan J. Watts and Steven H. Strogatz. 1998. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442.

Di Wu, Jia-Chen Gu, Kai-Wei Chang, and Nanyun Peng. 2025. Self-routing rag: Binding selective retrieval with knowledge verbalization. *arXiv preprint arXiv:2504.01018*.

Mingyan Wu, Han Yang, Omer Ben-Porat, and Yftah Ziser. 2026. When knowledge is not free: Cost-aware evidence selection in retrieval-augmented generation. *arXiv preprint arXiv:2606.02245*.

Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. 2025. When to use graphs in RAG: A comprehensive analysis for graph retrieval-augmented generation. *arXiv preprint arXiv:2506.05690*.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2024. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.

Yilin Xiao, Junnan Dong, Chuang Zhou, Su Dong, Qian-Wen Zhang, Di Yin, Xing Sun, and Xiao Huang. 2025. GraphRAG-Bench: Challenging domain-specific reasoning for evaluating graph retrieval-augmented generation. *arXiv preprint arXiv:2506.02404*.

Jiashu Yang, Chi Zhang, Abudukelimu Wuerkaixi, Xuxin Cheng, Cao Liu, Ke Zeng, Xu Jia, and Xunliang Cai. 2026. Autothinkrag: Complexity-aware control of retrieval augmented generation. *arXiv preprint arXiv:2603.05551*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.

Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Weichuan Liu, Lei Hou, and Juanzi Li. 2024. Seakr: Self-aware knowledge retrieval for adaptive retrieval augmented generation. *arXiv preprint arXiv:2406.19215*.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546.

Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 201–206.

Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. 2025. RAGRouter: Learning to route queries to multiple retrieval-augmented language models. *arXiv preprint arXiv:2505.23052*.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTscore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR 2020)*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36.

Appendix

A Data Construction & Corpus Analysis

A.1 Corpus Statistics & Preprocessing

Data Overview. To establish a benchmark encompassing diverse retrieval environments, we integrate four representative datasets spanning encyclopedic knowledge (MuSiQue, 21,100 Wikipedia articles), long-form narratives (QuALITY, 265 Gutenberg novels), specialized legal corpora (UltraDomain_Legal, 94 contract documents), and medical literature (GraphRAGBench_Medical, a single comprehensive textbook). As shown in Table 5, these datasets exhibit extreme disparities in both document count (ranging from 1 to 21,100) and average length (107.9 to 221,495 tokens), thereby serving as an ideal testbed for evaluating the adaptability of RAG paradigms across distinct scales and structural settings. The complete preprocessing pipeline is formalized in Algorithm 1.

Chunking Strategy. We employ a sliding window chunking strategy to process the raw corpora (see Table 6 for all hyperparameters). Specifically, utilizing the `cl100k_base` encoder from `tiktoken`, each document (concatenated title and content) is segmented into fixed-size chunks with a window size of 512 tokens and a 100-token overlap to preserve contextual coherence. This configuration strikes a balance between retrieval granularity and contextual completeness. Following this segmentation, the four datasets yield distinct chunk inventories: 21,153 (MuSiQue), 3,822 (QuALITY), 11,632 (Legal), and 538 (Medical).

Knowledge Graph Construction. To facilitate the structure-aware retrieval required by GraphRAG and HybridRAG, we extract knowledge triplets from each individual text chunk. We employ DeepSeek-V3 as the underlying extraction engine, with hyperparameters detailed in Table 6, setting the temperature to 0.0 to ensure deterministic generation. To optimize processing throughput, we implement an asynchronous parallelization strategy configured with a maximum concurrency of 15, a 60-second request timeout, and a retry mechanism allowing up to 3 attempts with exponential backoff. As illustrated in Figure 8, the extraction prompt instructs the model to identify `(Subject, Relation, Object)` triplets within the input text and format the output as a JSON array. The scale of the resulting knowledge graphs is detailed in Table 11. Specifically, MuSiQue comprises 206,738 entity nodes and 276,898 edges; QuALITY contains 90,088 nodes and 120,611 edges; the Legal corpus yields 135,231 nodes and 261,207 edges; and the Medical textbook generates 14,712 nodes and 21,480 edges. Graph density varies significantly from 6.0×10^{-6} (MuSiQue) to 9.9×10^{-5} (Medical), reflecting inherent disparities in structural sparsity across domains. All constructed graphs are represented as undirected graphs to facilitate bidirectional traversal.

Triplets Extraction

- Instructions:
You are an expert Knowledge Graph Engineer. Your task is to extract structured triplets (Subject, Relation, Object) from the provided text.
1. Coreference Resolution (MUST DO): You MUST resolve pronouns (he, she, it, they, the company) to their specific canonical names.
- BAD: ["He", "founded", "it"]
- GOOD: ["Elon Musk", "found", "SpaceX"]
- If a pronoun refers to an entity mentioned earlier in the text, use the Entity Name.
2. Entity Normalization:
- Use the most complete name: "NYC" -> "New York City", "Apple" -> "Apple Inc."
- Split compound entities: "Steve Jobs and Wozniak" -> Extract two separate triplets.
- Keep entities atomic: Extract "New York City" not "the city of New York"
3. Relation Normalization:
- Use active, base-form verbs: "was born in" -> "born_in", "is composed of" -> "comprise"
- Keep relations simple and snake_case
- Use lowercase for all relations
4. Edge Cases:
- Multiple subjects: "Jobs and Wozniak founded Apple" -> Create two triplets
- Nested relationships: Extract all levels (e.g., both "Paris located_in France" and "Eiffel Tower located_in Paris")
- Negations: Include if semantically important ("does not contain", "is not part of")

- Examples:
1. Temporal + Coreference:
- Input: "Steve Jobs co-founded Apple in 1976. He was later fired from the company but returned to lead it."
- Output: [["Steve Jobs", "co_found", "Apple Inc."], ["Apple Inc.", "founded_in", "1976"], ["Steve Jobs", "was_fired_from", "Apple Inc."], ["Steve Jobs", "lead", "Apple Inc."]]
2. Pronoun Resolution:
- Input: "The medication Aspirin is used to treat headaches. It can also reduce fever."
- Output: [["Aspirin", "treat", "headache"], ["Aspirin", "reduce", "fever"]]
3. Nested Relations:
- Input: "Paris is the capital of France and is known for the Eiffel Tower."
- Output: [["Paris", "is_capital_of", "France"], ["Paris", "known_for", "Eiffel Tower"], ["Eiffel Tower", "locate_in", "Paris"]]

Dataset	Domain	Num. Docs	Avg. Tokens	Total Tokens
MuSiQue	Wikipedia	21,100	107.9	2,276,013
QuALITY	Narrative	265	5,741.1	1,521,395
UltraDomain_legal	Legal	94	50,785.0	4,773,793
GraphRAGBench_medical	Medical Textbook	1	221,495.0	221,495

Table 5: **Raw Corpus Statistics.** Overview of the source documents illustrating extreme variations in document length (Avg. Tokens) and scale (Num. Docs), ranging from fragmented Wikipedia articles to monolithic textbooks.

Stage	Parameter	Value	Description
Chunking	Size	512	Fixed-size segment
	Overlap	100	Sliding window
	Tokenizer	cl100k_base	OpenAI encoding
Extraction	LLM	DeepSeek-V3	Base Model
	Temp.	0.0	Deterministic
	Concur.	15	Parallel requests
	Timeout	60s	API limit
	Retries	3	Fault tolerance
Graph	Directed	False	Undirected edges
Embedding	Model	text-emb-3-small	OpenAI Model
	Dim.	1536	Vector size
	Batch	30	API batch size

Table 6: Hyperparameters for Corpus Preprocessing.

4. Multiple Entities:
- Input: "Einstein and Bohr debated quantum mechanics in the 1920s."
- Output: [["Albert Einstein", "debate_with", "Niels Bohr"],
["Albert Einstein", "research_topic", "quantum mechanics"],
["Niels Bohr", "research_topic", "quantum mechanics"],
["quantum mechanics", "debated_in", "1920s"]]

- Output Format:
Return ONLY a valid JSON list of lists. No markdown, no explanations.

Figure 8: The prompt template for extracting structured knowledge triplets from text chunks.

Vectorization & Indexing. To facilitate the dense retrieval mechanism of NaiveRAG, we utilize the OpenAI `text-embedding-3-small` model to encode each text chunk into a 1,536-dimensional vector representation. We employ a batch size of 30 to mitigate API rate-limiting constraints. All generated vectors undergo L2 normalization and are subsequently indexed using `FAISS` (`IndexFlatIP`) to enable efficient retrieval based on cosine similarity. Concurrently, to support the entity-level retrieval required by GraphRAG, we generate distinct embeddings for all unique entities within the knowledge graphs, specifically, 206,738 for MuSiQue, 90,088 for QuALITY, 135,231 for Legal, and 14,712 for Medical. These entity embeddings serve as the foundation for precise entity matching and the selection of seed nodes for graph traversal during the query execution phase.

A.2 Query Generation Pipeline

Generation Overview. To construct a query set encompassing varying degrees of cognitive complexity, we devise three distinct query generation strategies, as detailed in Algorithm 2. All generation processes utilize DeepSeek-V3 as the backbone model, with a temperature setting of 0.7 to strike a balance between diversity and quality.

Factual Queries: We perform uniform random sampling of text segments from the corpus and employ the prompt illustrated in Figure 9 to steer the LLM in generating factual QA pairs. Crucially, the prompt enforces a *self-contained* constraint, ensuring that questions are semantically independent of context and devoid of ambiguous pronominal references.

Factual Query Generation

- Instructions:
1. The question should be answerable by a short phrase or sentence from the passage.
2. The answer must be explicitly stated in the passage.

Algorithm 1 Unified Corpus Preprocessing Pipeline

Require: Raw Corpus $\mathcal{C} = \{d_1, d_2, ..., d_n\}$, where each $d_i = (\text{title}, \text{text})$
Ensure: Knowledge Graph $\mathcal{G} = (V, E)$, Dense Vector Index $\mathcal{I}_{\text{vec}}$, Entity Embeddings $\mathbf{E}_{\text{ent}}$
Hyperparameters: Chunk Size $L = 512$, Overlap $O = 100$, Embedding Model $\mathcal{M}_{\text{emb}}$, LLM $\mathcal{M}_{\text{gen}}$
// Phase 1: Sliding Window Chunking
$\mathcal{S}_{\text{chunks}} \leftarrow \emptyset$
for each document $d \in \mathcal{C}$ **do**
  Tokens $\leftarrow$ Tokenize(d.title $\oplus$ d.text) ▷ Using `tiktoken` (cl100k_base)
  $ptr \leftarrow 0$
  while $ptr <$ len(Tokens) **do**
    $c_{\text{raw}} \leftarrow$ Tokens$[ptr : ptr + L]$
    $c_{\text{text}} \leftarrow$ Decode(c_{raw})
    $\mathcal{S}_{\text{chunks}} \leftarrow \mathcal{S}_{\text{chunks}} \cup \{(d.\text{id}, c_{\text{text}})\}$
    $ptr \leftarrow ptr + (L - O)$
  end while
end for
// Phase 2: Graph Construction & Entity Extraction
Initialize $V \leftarrow \emptyset, E \leftarrow \emptyset$
for each chunk $c \in \mathcal{S}_{\text{chunks}}$ **do**
  $\mathcal{T} \leftarrow \mathcal{M}_{\text{gen}}(\text{Prompt}_{\text{extract}}, c)$ ▷ Extract triplets (s, r, o) via DeepSeek
  for each triplet $(s, r, o) \in \mathcal{T}$ **do**
    $V \leftarrow V \cup \{s, o\}$
    $E \leftarrow E \cup \{(s, r, o)\}$
  end for
end for
// Phase 3: Vectorization & Indexing
$\mathbf{X}_{\text{chunks}} \leftarrow \emptyset$
for each chunk $c \in \mathcal{S}_{\text{chunks}}$ **do**
  $\mathbf{v}_c \leftarrow \mathcal{M}_{\text{emb}}(c)$ ▷ Dimension $d = 1536$
  $\mathbf{X}_{\text{chunks}}$.append($\mathbf{v}_c$)
end for
$\mathcal{I}_{\text{vec}} \leftarrow$ FAISS.Index($\mathbf{X}_{\text{chunks}}$) ▷ Build dense retrieval index
$\mathbf{E}_{\text{ent}} \leftarrow$ EmbedEntities(V) ▷ Embed unique entities for GraphRAG
return $\mathcal{G}, \mathcal{I}_{\text{vec}}, \mathbf{E}_{\text{ent}}$

3. Avoid yes/no questions.
4. The question must be understandable WITHOUT seeing the passage.
  - BAD: "When was he born?" (Who is 'he'?)
  - GOOD: "When was Barack Obama born?" (Specific Entity Name)
  - Action: You MUST replace pronouns (he, she, it, they) with the actual entity names found in the text.

- Output Format:
{"question": "The self-contained question", "answer": "The extractable answer"}

Figure 9: The prompt template for generating Factual queries.

Reasoning Queries: Leveraging the topological structure of the knowledge graph, we identify document chains linked via shared entities. Specifically, for k-hop inquiries, we execute random walks on the graph to locate k documents connected by bridge entities. Figure 10 depicts the generation prompt, which centers on a Reverse Substitution strategy: starting from the target answer, bridge entities are iteratively replaced with functional descriptions derived from preceding documents. This mechanism ensures that the resulting questions necessitate traversing the complete reasoning chain for resolution. We generate reasoning queries at both 2-hop and 3-hop complexity levels.

Reasoning Query Generation

- Instructions:
Generate a strict {num_hops}-hop question based on the provided {num_docs}-document chain.

- Generation Guidelines:
1. Analyze: Identify the Start Entity (Doc 1), Bridge Entities (Intermediate), and Final Answer (Last Doc).
2. Recursive Masking: Starting from the end, replace every Bridge Entity's name with a functional description derived strictly from the previous document.
  - Example: Instead of "Hawaii", use "the location where Jurassic Park was filmed".
3. Assembly: The final question must nest these descriptions and contain ONLY the specific name of the Start Entity.

- CRITICAL CONSTRAINTS:
1. Forbidden: NEVER use the actual name of any Bridge Entity in the question.
2. Dependency: The question must become unanswerable if any single document is removed.
3. Answer Isolation: The Final Answer must NOT appear in any document except the last one.
4. Grammar: Ensure natural phrasing for nested structures.

- Output Format:
{{
  "reasoning": "Step 1: Identify Bridge [Name] and mask it as '[Description]'; Step 2: Identify Bridge [Name] and mask it as '[Description]'; Step 3: Verify strict dependency.",
  "question": "The final nested question",
  "answer": "The final answer from the last Doc"
}}

Figure 10: The instruction set for synthesizing Reasoning queries.

Summary Queries: We cluster documents by entity, selecting those with high connectivity within the graph as summarization targets. As shown in Figure 11, the prompt mandates an initial *consistency check*, verifying that multiple documents refer to the same entity rather than homonyms, followed by the synthesis of information from at least two documents. This process yields summarization questions that explicitly require cross-document integration for a complete response.

Summary Query Generation

- Instructions:
You are an expert dataset creator.
Your task is to generate a Synthesis Question and a Comprehensive Answer based on a cluster of documents related to the entity: "{entity}".

- Generation Guidelines:
1. Consistency Check: Do these documents talk about the SAME "{entity}"?
- If one doc is about "Michael Jordan (Basketball)" and another about "Michael Jordan (Professor)", return {{"discard": true}}.
2. Information Synthesis (The Answer):
- Identify common themes across the documents (e.g., Career, Impact, Controversy, Evolution).
- Write a high-quality summary (3-5 sentences) that integrates facts from AT LEAST 2 different documents.
- Constraint: Do not simply list facts. Connect them (e.g., "While Doc A mentions X, Doc B clarifies that Y...").
3. Question Formulation (The Query):
- Write a question that naturally elicits the summary you just wrote.
- Specific Theme: Instead of "Summarize X", ask about a specific aspect like "How did X's career evolve?" or "What are the key contributions of X?".
- Self-Contained: The question MUST contain the full name of "{entity}". Do NOT use pronouns like "he/she" or vague phrases like "the provided text".
- BAD: "Summarize his achievements mentioned in the text."
- GOOD: "What were the major scientific achievements of Albert Einstein?"

- Output Format:
{{ "reasoning": "Step 1: Themes identified are... Step 2: Documents used are Doc [x] and Doc [y]...",
"question": "The self-contained synthesis question",
"answer": "The synthesized answer covering multiple docs"
}} OR {{"discard": true}}

Figure 11: The prompt designed for Summary query generation.

Verify-then-Filter Validation. Raw queries undergo a rigorous three-tiered verification protocol before inclusion in the final benchmark (Algorithm 2, Phase 2).

Grounding Check: We task an LLM with generating answers derived exclusively from the supporting facts. An LLM-as-a-Judge then validates the semantic consistency between the generated response and the expected answer. This step guarantees that the question is rigorously *answerable* given the provided documents.

Shortcut Detection: We evaluate whether any single supporting fact suffices to answer the question in isolation. If an individual document yields the correct answer, the query is identified as containing a "shortcut", violating the intrinsic requirement of multi-hop reasoning, and is subsequently filtered out.

Knowledge Leakage Check: We screen for two forms of information leakage: (1) Lexical Leakage, where the answer appears as a substring within the question itself; and (2) Parametric Leakage, where the LLM can answer correctly relying solely on pre-trained knowledge. The latter is assessed via a closed-book test; if the model succeeds without retrieval context, the query is deemed ineffective for evaluating retrieval capabilities.

Human Verification: To address potential LLM self-validation bias in the above automated checks, we conduct manual verification on a stratified sample of 50 queries per corpus (N=200 total). Two annotators independently assess answerability and information leakage, achieving 91% inter-annotator agreement (Cohen's κ=0.85). The human-LLM agreement rate of 94% confirms the reliability of the automated filtering pipeline.

Validation Statistics. Table 9 presents the validation statistics across the constituent datasets. We generated a total of 7,444 candidate queries, of which 3,414 were retained following the *Verify-then-Filter* process, yielding an overall acceptance rate of 45.9%.

Benchmark Composition. Table 7 consolidates the full provenance of the 7,727 benchmark queries, distinguishing queries inherited from the source benchmarks (*native*) from those produced by our augmentation pipeline, and reporting the final query-type distribution per corpus. MuSiQue and GraphRAGBench-Medical contribute native queries, while QuALITY and UltraDomain-Legal are populated entirely through chunk-grounded augmentation, since neither source provides queries covering all three canonical types.

Query Statistics. Table 8 reports surface statistics of the final query set. Query length varies substantially across corpora, from short lookup questions in the Medical textbook (12.5 tokens on average) to longer descriptive questions over legal contracts (31.6 tokens), reflecting the distinct question styles that each domain induces. Verbatim duplicate rates are below 0.4% on QuALITY, Legal, and Medical. The higher rate on MuSiQue (2.831%) originates from paraphrased questions already present in the native MuSiQue split rather than from augmentation.

Pass rates exhibit significant variance across distinct query types. Single-hop queries achieve

Corpus	Provenance				Query Type		
	Generated	After filter	Native	**Total**	Factual	Reasoning	Summary
MuSiQue	2,027	939	2,417	3,356	398	2,590	368
QuALITY	2,450	1,198	0	1,198	454	461	283
UltraDomain-Legal	2,967	1,277	0	1,277	370	526	381
GraphRAGBench-Medical	0	0	1,896	1,896	1,098	509	289
Total	7,444	3,414	4,313	**7,727**	2,320	4,086	1,321

Table 7: **Benchmark composition and query provenance.** "Generated" counts candidate queries produced by the augmentation pipeline; "After filter" counts those surviving the verify-then-filter protocol (overall acceptance rate 45.9%); "Native" counts queries inherited from the source benchmark.

Corpus	Avg. len.	Std.	Dup. rate
MuSiQue	19.0	9.1	2.831%
QuALITY	25.4	15.5	0.000%
UltraDomain-Legal	31.6	19.8	0.392%
GraphRAGBench-Medical	12.5	4.3	0.316%

Table 8: **Query surface statistics.** Length in `cl100k_base` tokens. "Dup. rate" is the proportion of verbatim duplicate queries within each corpus.

pass rates exceeding 90% on QuALITY and Legal datasets, yet only 56.9% on MuSiQue. This disparity is primarily attributed to MuSiQue's foundation in Wikipedia, where extensive factual overlap with the LLM's pre-training corpus frequently triggers the Knowledge Leakage filter. Multi-hop queries register the lowest pass rates (21%–42%), with the vast majority discarded by Shortcut Detection, underscoring the inherent challenge in generating questions that genuinely necessitate multi-step reasoning. Summary queries exhibit pass rates ranging from 36% to 70%, with failures predominantly stemming from the Grounding Check, specifically, semantic deviations between the response derived from the provided document set and the expected gold standard.

After the verify-then-filter protocol, 3,414 of the 7,444 generated queries are retained. Together with the 4,313 native queries inherited from MuSiQue and GraphRAGBench-Medical, the benchmark comprises 7,727 queries spanning four domains and three levels of cognitive complexity (Table 7).

A.3 Corpus Evaluation Metrics

Table 10 summarizes the core metrics employed to characterize the retrieval environment. By quantifying corpus properties along the dual dimensions of *structural topology* and *semantic space*, these metrics provide a quantitative foundation for delineating the applicability boundaries of distinct RAG paradigms.

Structural Topology Metrics. We employ three graph-theoretic metrics to evaluate the topological structure of the knowledge graphs (Table 11):

LCC Ratio (Largest Connected Component Ratio): This metric measures global reachability. It is defined as the ratio of nodes in the largest connected component to the total number of nodes:

$$\text{LCC Ratio} = \frac{|V_{\text{LCC}}|}{|V|} \tag{1}$$

where V_{LCC} denotes the node set of the largest connected component and V represents the total node set. A lower ratio indicates severe graph fragmentation, increasing the risk that multi-hop reasoning paths are physically severed.

Density: This metric measures edge saturation. For an undirected graph, it is defined as:

$$D = \frac{2|E|}{|V|(|V|-1)} \tag{2}$$

where $|E|$ is the number of edges and $|V|$ is the number of nodes. Excessively low density implies a lack of sufficient relational bridges between entities, while excessively high density introduces noise, thereby degrading graph traversal efficiency.

Clustering Coefficient: This metric measures local cohesiveness. The clustering coefficient for a node v is defined as the ratio of actual edges between its neighbors to the number of possible edges:

$$C_v = \frac{2 \cdot |\{e_{jk} : v_j, v_k \in N(v), e_{jk} \in E\}|}{k_v(k_v-1)} \tag{3}$$

where $N(v)$ is the neighborhood set of node v, and $k_v = |N(v)|$ represents the node degree. The global clustering coefficient is the average over all nodes. A high coefficient indicates the presence of tight-knit thematic communities, facilitating local evidence aggregation.

Type	MuSiQue			QuALITY			Legal		
	Gen.	Pass	Rate	Gen.	Pass	Rate	Gen.	Pass	Rate
Single-hop	700	398	56.9%	500	454	90.8%	400	370	92.5%
2-hop	400	84	21.0%	561	212	37.8%	784	238	30.4%
3-hop	400	89	22.2%	600	249	41.5%	800	288	36.0%
Summary	527	368	69.8%	789	283	35.9%	983	381	38.8%
Dataset Total	*2,027*	*939*	*46.3%*	*2,450*	*1,198*	*48.9%*	*2,967*	*1,277*	*43.0%*
Overall Total							**7,444**	**3,414**	**45.9%**

Table 9: Verify-then-Filter validation statistics.

Dimension	Metric	Physical Interpretation	Constraint Mechanism on Retrieval
Structural	LCC Ratio	Global Reachability	Fragmentation into isolated components severs reasoning paths for multi-hop retrieval.
	Density	Edge Saturation	Sparse graphs lack sufficient relational bridges; overly dense graphs introduce noise.
	Clustering Coeff.	Local Coherence	Facilitates evidence aggregation within topical neighborhoods.
Semantic	Intrinsic Dim.	Effective Degrees of Freedom	High dimensionality exacerbates the curse of dimensionality, degrading similarity metrics.
	Dispersion	Semantic Spread	Low dispersion causes semantic crowding, hindering distinction of hard negatives.
	Hubness	Retrieval Interference	Hub embeddings dominate neighbor lists, causing systematic retrieval bias.

Table 10: Key corpus metrics characterizing the retrieval environment.

Dataset	Nodes	Edges	Density	Rel. Types	Avg. Deg.	Comp.	LCC Ratio	Cluster. Coeff.
MuSiQue	206,738	276,898	6.00e-06	44,766	2.68	7,722	0.882	0.0213
QuALITY	90,088	120,611	1.50e-05	23,828	2.68	3,997	0.883	0.0177
Legal	135,231	261,207	1.40e-05	28,799	3.86	3,204	0.933	0.0701
Medical	14,712	21,480	9.90e-05	4,169	2.92	741	0.861	0.0357

Table 11: **Full Structural Statistics.** Detailed graph topology metrics including node/edge counts, graph density, number of unique relation types, average node degree, number of connected components (Comp.), ratio of the largest connected component (LCC Ratio), and average clustering coefficient.

Semantic Space Metrics We employ three vector space metrics to assess embedding quality (Table 12):

Intrinsic Dimension: This metric quantifies the effective degrees of freedom within the embedding space. We estimate it using the **TwoNN** algorithm: for each data point, we calculate the distance to its nearest neighbor (r_1) and second-nearest neighbor (r_2). Letting $\mu = r_2/r_1$, the intrinsic dimension is defined as:

$$d_{\text{int}} = \frac{1}{\mathbb{E}[\ln \mu]} \tag{4}$$

High intrinsic dimensionality exacerbates the *curse of dimensionality*, rendering distance-based similarity metrics ineffective.

Dispersion: This metric measures the uniformity of semantic distribution. We compute the cosine distance of each embedding vector from the global centroid:

$$\text{dist}(x_i) = 1 - \frac{x_i \cdot \bar{x}}{\|x_i\|\|\bar{x}\|} \tag{5}$$

where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ denotes the centroid vector. Table 12 reports the mean, standard deviation, minimum, and maximum of these distances. Low dispersion results in *semantic crowding*, hindering the retriever's ability to distinguish between semantically similar yet factually unrelated "hard negatives."

Hubness: This metric quantifies the extent of retrieval interference. Defining $N_k(i)$ as the number of times point i appears in the k-nearest neighbor lists of all other points (k-occurrence), Hubness is calculated as the skewness of this distribution:

$$S_k = \frac{\mathbb{E}[(N_k - \mu_{N_k})^3]}{\sigma_{N_k}^3} \tag{6}$$

Dataset	Chunks	Int. Dim.	Hubness	Avg. Dist.	Std. Dist.	Min. Dist.	Max. Dist.
MuSiQue	21,153	8.17	1.27	0.708	0.049	0.552	0.924
QuALITY	3,822	10.75	1.26	0.345	0.119	0.186	0.805
Legal	11,632	7.56	1.46	0.300	0.071	0.147	0.792
Medical	538	8.39	0.86	0.312	0.063	0.196	0.700

Table 12: **Full Semantic Statistics.** Metrics covering vector space properties: total number of text chunks, intrinsic dimension (Int. Dim.), Hubness score (interference), and centroid distance statistics (Average, Standard Deviation, Minimum, Maximum).

Positive skewness indicates the presence of "hub" embeddings, vectors that frequently appear in the nearest neighbor lists of others. This phenomenon causes a systematic bias in retrieval results towards these hubs, thereby reducing both retrieval diversity and accuracy.

B RAG Paradigm Implementation

The experimental setup of Section 3.4 is documented across the next three sections, each covering a different object. This section covers the paradigms themselves, their shared and paradigm-specific hyperparameters and their retrieval pipelines. Appendix C covers how generated answers are measured, including the metric implementations, the judge and the cost decomposition. Appendix D covers the routers, including their features, training protocol and evaluation metrics. The architectural overview is illustrated in Figure 12.

B.1 Unified Hyperparameters

To guarantee a fair comparison across distinct RAG paradigms, we standardize the core hyperparameter configurations. The two tables below divide the settings: Table 13 fixes everything the paradigms hold in common, so that any performance difference is attributable to the retrieval strategy rather than to the generator or the budget, while Table 14 gives the parameters that distinguish them. Both generation backbones, DeepSeek-V3 and LLaMA-3.1-8B, are decoded at temperature 0.3 to elicit stable outputs, and every retrieval-augmented paradigm is constrained by a shared context budget of 8,000 tokens, employs cosine similarity as the distance metric, and enforces a minimum relevance threshold of 0.4 to filter out low-quality evidence.
Table 14 details the paradigm-specific parameters. LLM-only, serving as the retrieval-free baseline, utilizes a higher temperature (0.7) to encourage the model to fully leverage its internal parametric knowledge. NaiveRAG retrieves a maximum of 100 text chunks, truncating the selection based on similarity ranking to fit within the token budget. GraphRAG initiates from 20 seed entities and expands the subgraph using Personalized PageRank ($\alpha = 0.85$), retaining a maximum of 100 nodes and 500 triplets. IterativeRAG uses NaiveRAG as its base retriever in the reported results, with a GraphRAG-based variant reported separately in Table 18, executing up to 3 rounds of iterative refinement, with the evaluator operating at a low temperature (0.1) to ensure decisional consistency. Finally, HybridRAG inherits the parameter settings of both NaiveRAG and GraphRAG, fusing their respective retrieval results.

B.2 Retrieval Paradigm Implementation

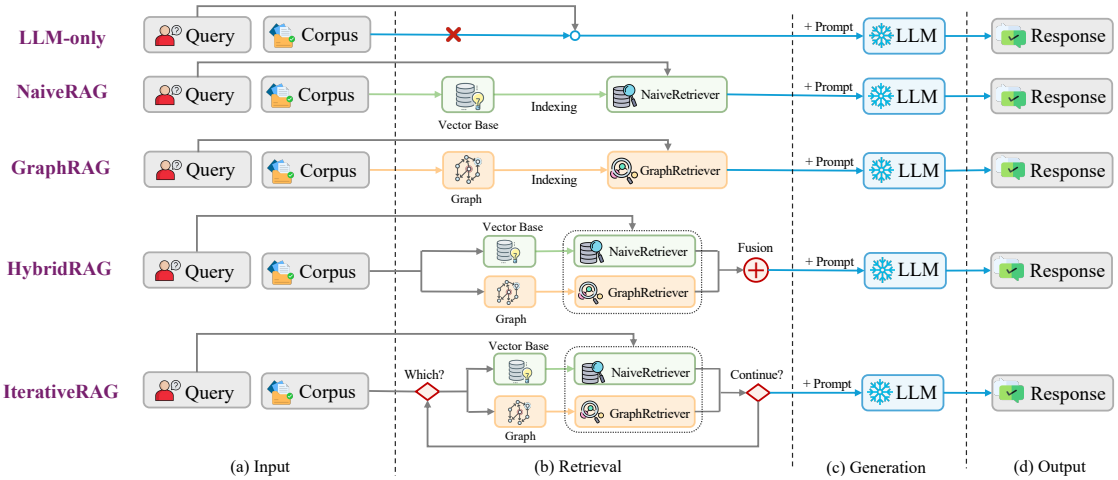
Table 15 contrasts the five paradigms from a methodological perspective, characterizing their retrieval substrates, information granularity, and search mechanisms. This section elaborates on the specific implementation details of each paradigm. To ensure a fair comparison, all methodologies employ a uniform generation prompt, as illustrated in Figure 13.

Retrieval Answer Generation

- Instructions:
You are a strict and accurate Answer Generator based on retrieved context.
1. Context-Driven: Answer the question using ONLY the provided context snippets. Do not use your own internal knowledge.
2. Refusal: If the provided context does not contain the information needed to answer the question, output EXACTLY: "I cannot answer" (no explanation, no reasoning).
3. No Meta-talk: Do not say "According to the context..." or "The document says...". Just give the answer.

- Adaptive Answer Format:
1. Specific Fact Extraction (e.g., Who, When, Where)
- Goal: Single-hop / Factual
- Format: Provide the precise entity, date, number, or name. Be atomic and concise.
- Example: "Steve Jobs" (NOT "Steve Jobs is the CEO.")
2. Complex Reasoning (e.g., Comparative, Multi-step)

Figure 12: **Overview of the five RAG paradigms evaluated in RAGRouter-Bench. (a) Input:** Query and corpus shared across all paradigms. **(b) Retrieval:** Paradigm-specific pipelines. **(c) Generation:** Retrieved context combined with query as prompt to LLM. **(d) Output:** Final response.



Category	Parameter	Value	Description
LLM	Model	DeepSeek-V3 / LLaMA-3.1-8B	Generation backbones
	Temperature	0.3	Focused generation
	Max Tokens	1000	Response length limit
	Timeout	120s	API request limit
Retrieval	Token Budget	8000	Context length limit
	Similarity	Cosine	Distance metric
	Min Threshold	0.4	Relevance filter

Table 13: Shared hyperparameters across all RAG paradigms.

Paradigm	Parameter	Value	Description
LLM-only	Temperature	0.7	Creative generation
	Max Tokens	1000	Response length
NaiveRAG	Top-K	100	Max chunks retrieved
GraphRAG	Seed Entities	20	Initial anchors
	PPR Alpha	0.85	Damping factor
	PPR Max Nodes	100	Subgraph size limit
	Max Triplets	500	Serialization limit
HybridRAG	RRF k	60	Fusion smoothing constant
IterativeRAG	Base Retriever	NaiveRAG (reported) / GraphRAG	Underlying method
	Max Iterations	3	Reasoning loop limit
	Eval Temp.	0.1	Evaluator setting

Table 14: Paradigm-specific hyperparameters.

- Goal: Multi-hop
- Format: Perform the reasoning across the context and output the direct conclusion.
- Example: "Paris" (If asked for the capital of the country where X was born)

3. Overview or Explanation (e.g., Summarize, How, Why)
- Goal: Summarization
- Format: Synthesize information from multiple parts of the context into a coherent, comprehensive paragraph (3-5 sentences). Do not simply list bullet points unless requested.

- Final Check:
1. Does your answer directly address the prompt?
2. Is it fully supported by the text?

Figure 13: The Context-Aware Generation prompt.

Paradigm	Retrieval Substrate	Info. Granularity	Search Mechanism	Optimal Use Cases
LLM-only	Parametric Weights (Implicit)	Internal Knowledge	Next-token Prediction	General chit-chat; Creative writing; Tasks requiring no external facts (Mallen et al., 2023).
NaiveRAG	Flat Vector Space	Coarse-grained (Passage/Chunk)	Semantic Similarity (Dense Retrieval)	Explicit fact retrieval; Simple QA; Queries with high semantic overlap (Karpukhin et al., 2020).
GraphRAG	Graph Topology (Knowledge Graph)	Fine-grained (Entity/Relation)	Structure-aware Traversal	Multi-hop reasoning (Yasunaga et al., 2021); Global summarization (Edge et al., 2024); Connecting disparate information.
HybridRAG	Dual-Pathway Space (Vector + Graph)	Multi-granular (Chunk + Entity/Relation)	Reciprocal Rank Fusion (Vector + Graph)	Queries needing both semantic similarity and entity relations; Cases where either pathway alone under-retrieves (Sarmah et al., 2024).
IterativeRAG	Dynamic Context	Adaptive (Coarse to Fine)	Multi-step Feedback (Reasoning Loop)	Ambiguous queries; Complex research; Tasks needing progressive clarification (Trivedi et al., 2023; Asai et al., 2024).

Table 15: **A Methodology Perspective on RAG Paradigms.** We categorize existing paradigms by their *Retrieval Substrate* (data structure), *Information Granularity*, and *Search Mechanism*. Each paradigm imposes different trade-offs between retrieval cost and reasoning capability, highlighting that no single strategy fits all scenarios.

LLM-only and NaiveRAG. LLM-only operates as the retrieval-free baseline, generating responses by directly querying the model and thereby relying exclusively on its internal parametric knowledge (Figure 14). NaiveRAG follows the standard dense retrieval protocol, retrieving the top-100 semantically similar text chunks and subsequently truncating the concatenated context to adhere to the 8,000-token budget limit.

LLM-only Generation

- Instructions:
You are a helpful AI assistant evaluating your internal knowledge base.
Your task is to answer the user's question based PURELY on your pre-trained knowledge.
1. Internal Knowledge: Use your own memory to answer. Do not ask for context.
2. Conciseness: Provide the answer directly and concisely.
- If the answer is an entity, date, or number, output ONLY that specific string.
- Do not output full sentences like "The answer is..." unless necessary.
3. Honesty: If you do not know the answer or are unsure, reply EXACTLY with: "I cannot answer" (no explanation, no reasoning).
4. Do NOT hallucinate or make up facts.

- Examples:
1. Q: "What is the capital of France?"
A: Paris
2. Q: "Who is the director of the movie starring the lead actor of Titanic?"
A: Christopher Nolan
3. Q: "What is the specific diameter of the hidden pipe in the fictional factory of generic novel X?"
A: I cannot answer

Figure 14: The prompt for Direct Generation (LLM-only).

GraphRAG. GraphRAG leverages knowledge graphs to perform structure-aware retrieval (Algorithm 4). The process initiates by extracting query entities via an LLM (Figure 15) and linking them to graph entities to establish a seed node set S. Subsequently, Personalized PageRank (PPR) is executed over the global graph topology. We construct the personalization vector $\mathbf{p}$ based on semantic similarity to the seeds:

$$\mathbf{p}[v] = \frac{\text{sim}(v)}{\sum_{u \in S} \text{sim}(u)}, \quad v \in S \tag{7}$$

The PPR iterative update rule is defined as:

$$\boldsymbol{\pi}^{(t+1)} = \alpha \cdot \mathbf{A} \cdot \boldsymbol{\pi}^{(t)} + (1 - \alpha) \cdot \mathbf{p} \tag{8}$$

where $\mathbf{A}$ denotes the column-normalized adjacency matrix of the graph, and $\alpha = 0.85$ serves as the damping factor. Upon convergence, we identify the top-100 nodes with the highest PPR scores to construct a salient subgraph. The associated triplets are then extracted and mapped back to their original textual source to serve as the generation context.

HybridRAG. HybridRAG integrates the retrieval outputs from both NaiveRAG (vector-based) and GraphRAG (graph-based). Following the independent acquisition of ranked lists from both pathways, we employ Reciprocal Rank Fusion (RRF)

Algorithm 2 Query Generation & Validation Pipeline

Require: Corpus $\mathcal{C}$, Knowledge Graph $\mathcal{G}$, Target Counts $N_{\text{fact}}, N_{\text{hop}}, N_{\text{sum}}$
Ensure: Validated Query Set $\mathcal{Q}_{\text{final}}$
Hyperparameters: Generator $\mathcal{M}_{\text{gen}}$, Evaluator $\mathcal{M}_{\text{eval}}$
// Phase 1: Diversity-Driven Generation
$\mathcal{Q}_{\text{raw}} \leftarrow \emptyset$
(1) Factual Queries (Single-hop):
for $i \leftarrow 1$ **to** N_{fact} **do**
  $c \sim \text{Uniform}(\mathcal{C})$ ▷ Sample random chunk
  $(q, a) \leftarrow \mathcal{M}_{\text{gen}}(\text{Prompt}_{\text{fact}}, c)$
  $\mathcal{Q}_{\text{raw}} \leftarrow \mathcal{Q}_{\text{raw}} \cup \{(q, a, \{c\}, \text{"factual"})\}$
end for
(2) Reasoning Queries (Multi-hop):
$\mathcal{P}_{\text{chains}} \leftarrow \text{RandomWalk}(\mathcal{G}, \text{len} = 2)$ ▷ Find connected doc pairs via bridge entities
for $i \leftarrow 1$ **to** N_{hop} **do**
  $D_{\text{chain}} \leftarrow \mathcal{P}_{\text{chains}}[i]$
  $(q, a, \text{reasoning}) \leftarrow \mathcal{M}_{\text{gen}}(\text{Prompt}_{\text{hop}}, D_{\text{chain}})$
  $\mathcal{Q}_{\text{raw}} \leftarrow \mathcal{Q}_{\text{raw}} \cup \{(q, a, D_{\text{chain}}, \text{"multi_hop"})\}$
end for
(3) Summary Queries (Global):
for $i \leftarrow 1$ **to** N_{sum} **do**
  $e \sim \text{PageRank}(\mathcal{G})$ ▷ Sample important entity
  $D_{\text{cluster}} \leftarrow \text{GetNeighbors}(e, \mathcal{G})$ ▷ Retrieve ego-graph documents
  $(q, a) \leftarrow \mathcal{M}_{\text{gen}}(\text{Prompt}_{\text{sum}}, D_{\text{cluster}})$
  $\mathcal{Q}_{\text{raw}} \leftarrow \mathcal{Q}_{\text{raw}} \cup \{(q, a, D_{\text{cluster}}, \text{"summary"})\}$
end for
// Phase 2: The "Verify-then-Filter" Validation Loop
$\mathcal{Q}_{\text{final}} \leftarrow \emptyset$
for each query instance $\mathbf{x} = (q, a, D_{\text{supp}}, \text{type}) \in \mathcal{Q}_{\text{raw}}$ **do**
  valid $\leftarrow$ **True**
  Check 1: Grounding (Answerable from Context?)
  $\hat{a} \leftarrow \mathcal{M}_{\text{eval}}(\text{Prompt}_{\text{qa}}, q, D_{\text{supp}})$
  if $\text{Sim}(\hat{a}, a) < \tau_{\text{strict}}$ **then** valid $\leftarrow$ **False**
  end if
  Check 2: Shortcut Detection (Multi-hop Only)
  if type == "multi_hop" **then**
    for $d \in D_{\text{supp}}$ **do**
      if $\mathcal{M}_{\text{eval}}(q, \{d\}) \approx a$ **then** ▷ Can single doc answer it?
        valid $\leftarrow$ **False**; **break**
      end if
    end for
  end if
  Check 3: Knowledge Leakage (LLM Prior Knowledge)
  $a_{\text{prior}} \leftarrow \mathcal{M}_{\text{eval}}(\text{Prompt}_{\text{closed_book}}, q)$ ▷ Ask without context
  if $\text{Sim}(a_{\text{prior}}, a) > \tau_{\text{leak}}$ **then**
    valid $\leftarrow$ **False** ▷ Reject if LLM already knows the answer
  end if
  if valid **then** $\mathcal{Q}_{\text{final}} \leftarrow \mathcal{Q}_{\text{final}} \cup \{\mathbf{x}\}$
  end if
end for
return $\mathcal{Q}_{\text{final}}$

to merge the rankings:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{9}$$

where $\mathcal{R} = \{\text{Naive}, \text{Graph}\}$ represents the set of retrievers, $\text{rank}_r(d)$ denotes the rank position of document d within retriever r, and $k = 60$ serves as the smoothing constant. Post-fusion, documents are sorted in descending order of their RRF scores. We subsequently remove duplicates and truncate the sequence to adhere to the strict 8,000-token context budget.

IterativeRAG. IterativeRAG implements a "Retrieve-Generate-Evaluate" feedback loop, as detailed in Algorithm 3. In the initial phase (Round 0), the system attempts a direct response using the LLM. If the evaluator (Figure 16) deems this response insufficient, it generates targeted sub-questions to trigger the retrieval cycle. In each subsequent iteration, newly retrieved evidence is aggregated with the cumulative context to synthesize an updated answer, which is then re-evaluated. This cycle persists until one of the following termination criteria is met: (i) the answer is judged sufficient; (ii) the maximum iteration count ($T = 3$) is reached; (iii) no new sub-questions are generated; or (iv) generated sub-questions are repetitive. We instantiate the framework using either NaiveRAG or GraphRAG as the underlying base retriever.

Entity Extraction

- Instructions:
You are an expert at extracting named entities from text.
1. Identify all entities: people, organizations, locations, dates, events, concepts
2. Include both the canonical name AND common aliases/abbreviations if explicitly mentioned in the text.
  - Include aliases only if they refer to the same real-world entity.
  - Do NOT invent aliases that are not explicitly mentioned.
3. Resolve pronouns to their referent entity (he/she/it → the entity they refer to)
4. Exclude generic pronouns (it, this, that) that don't refer to a specific named entity.
5. Exclude common stop words and articles.

- Examples:
1. Text: "Tony Stark founded Apple. He later sold the company. NYC became its headquarters."
  Output: ["Tony Stark", "Apple", "NYC"]
2. Text: "Apple Inc., commonly known as Apple, was founded by Steve Jobs. The company is based in NYC (New York City)."
  Output: ["Apple Inc.", "Apple", "Steve Jobs", "NYC", "New York City"]

- Output Format:
Return ONLY a JSON list of entities:
["entity1", "entity2", ...]

Figure 15: The prompt template for Entity Extraction.

IterativeRAG Evaluation

- Instructions:
You are an answer evaluation assistant. Assess whether the answer adequately addresses the question.
Evaluate the given answer and determine if it sufficiently answers the question.
1. Set "sufficient" to true if the answer correctly and completely addresses the question
2. Set "sufficient" to false if the answer is wrong, incomplete, or refuses to answer (e.g., "I cannot answer")
3. If insufficient, generate a "sub_question" that targets the specific missing information needed to answer the original question
4. The "sub_question" should be a focused query suitable for document retrieval
5. If sufficient, set "sub_question" to null

- Output Format:
Respond ONLY in JSON with exactly these fields:
{"sufficient": true/false, "reason": "brief explanation", "sub_question": "follow-up question if insufficient, else null"}

Figure 16: The Self-Evaluation prompt for Iterative RAG.

LLM-as-a-Judge

- Instructions:
You are evaluating final answer correctness.
1. Do NOT consider writing quality, fluency, or style.
2. Focus only on whether the final conclusion is correct.
3. Treat the final conclusion as the specific answer that would fully resolve the question.
4. If the answer is partially correct, vague, or missing key information, choose incomplete.
5. If the answer gives a specific wrong conclusion, choose incorrect.
6. If the answer refuses to answer (e.g., "I cannot answer"), choose incomplete.
7. Classify the predicted answer into ONE of the following categories:
- correct: The answer gives the correct final conclusion required by the ground truth.
- incorrect: The answer gives a wrong final conclusion that conflicts with the ground truth.
- incomplete: The answer attempts to answer but does not give the correct final conclusion (includes refusal to answer).

- Examples:
1. GT: "Paris", Pred: "The capital of France is Paris" → correct
2. GT: "1976", Pred: "Apple was founded in 1975" → incorrect
3. GT: "Steve Jobs", Pred: "I cannot answer based on the context" → incomplete

- Output Format:
Respond ONLY in JSON with exactly these fields:
{"label": "<correct/incorrect/incomplete>", "reason": "<one short sentence explaining why>"}

Figure 17: The LLM-as-a-Judge instruction template used for automated evaluation.

Algorithm 3 IterativeRAG (Multi-Round Retrieval with Self-Evaluation)

Require: Question q, Base Retriever $\mathcal{R}$ (NaiveRAG or GraphRAG), Max Iterations T
Ensure: Final Answer a

Initialization
$\mathcal{Q}_{\text{history}} \leftarrow \{q\}$ ▷ Track all queries to prevent loops
$\mathcal{C}_{\text{accum}} \leftarrow \emptyset$ ▷ Accumulated retrieved chunks
$H \leftarrow []$ ▷ Reasoning trace

Round 0: Direct LLM Answer (No Retrieval)
$a_0 \leftarrow \text{LLM}(\text{Prompt}_{\text{QA}}, q)$ ▷ Answer without context
$\text{eval}_0 \leftarrow \text{LLM}(\text{Prompt}_{\text{Eval}}, q, a_0)$ ▷ {sufficient, reason, sub_question}
$H.\text{append}((0, q, a_0, \text{eval}_0))$
if eval_0.sufficient **is True then**
  return a_0 ▷ LLM already knows the answer
end if
$q_{\text{curr}} \leftarrow \text{eval}_0.\text{sub_question}$ **or** q ▷ Get refined query

Round 1+: Iterative Retrieval Loop
for $t \leftarrow 1$ **to** T **do**
// Step 1: Retrieve new chunks
  $\mathcal{C}_{\text{new}} \leftarrow \text{Retrieve}(\mathcal{R}, q_{\text{curr}})$
// Step 2: Merge and Deduplicate
  $\mathcal{C}_{\text{accum}} \leftarrow \mathcal{C}_{\text{accum}} \cup \mathcal{C}_{\text{new}}$
// Step 3: Apply Token Budget
  $\mathcal{C}_{\text{ctx}} \leftarrow \text{ApplyTokenBudget}(\mathcal{C}_{\text{accum}}, B = 8000)$
  $\text{ctx} \leftarrow \text{Concatenate}(\mathcal{C}_{\text{ctx}})$
// Step 4: Generate answer with accumulated context
  $a_t \leftarrow \text{LLM}(\text{Prompt}_{\text{RAG}}, \text{ctx}, q)$ ▷ Always answer ORIGINAL question
// Step 5: Evaluate answer sufficiency
  $\text{eval}_t \leftarrow \text{LLM}(\text{Prompt}_{\text{Eval}}, q, a_t, \text{ctx})$
  $H.\text{append}((t, q_{\text{curr}}, a_t, \text{eval}_t))$
// Step 6: Check termination conditions
  if eval_t.sufficient **is True then**
    return a_t ▷ Answer is sufficient
  end if
  $q_{\text{next}} \leftarrow \text{eval}_t.\text{sub_question}$
  if q_{next} **is null then**
    return a_t ▷ No further refinement possible
  end if
  if $q_{\text{next}} \in \mathcal{Q}_{\text{history}}$ **then**
    return a_t ▷ Prevent query loop
  end if
// Step 7: Update for next iteration
  $\mathcal{Q}_{\text{history}} \leftarrow \mathcal{Q}_{\text{history}} \cup \{q_{\text{next}}\}$
  $q_{\text{curr}} \leftarrow q_{\text{next}}$
end for
return $H[-1]$.answer ▷ Return last answer if max iterations reached

C Evaluation & Analysis Details

C.1 Metric Implementation

Metric Categories. To comprehensively assess the generation quality of RAG systems, we devise a multi-dimensional evaluation framework (Table 17). The metrics fall into two classes. Answer Quality metrics quantify how well the generated response matches the gold standard, covering LLM-as-a-Judge for holistic correctness aligned with human judgment, Semantic F1 for token-level similarity, and coverage for informational completeness.

Method	Type	MuSi.			QuAL.			Legal			Med.		
		Cor	Inc	No-A	Cor	Inc	No-A	Cor	Inc	No-A	Cor	Inc	No-A
Model: DeepSeek-V3													
NaiveRAG	Multi-hop	28.3	24.0	47.7	33.8	48.8	17.4	11.2	13.3	75.5	63.5	5.1	31.4
	Single-hop	11.1	5.3	83.7	**83.7**	13.2	3.1	54.9	17.3	27.8	63.1	6.2	30.7
	Summary	29.4	27.5	43.2	17.0	30.7	52.3	39.1	11.6	49.3	49.1	1.0	49.8
GraphRAG	Multi-hop	22.5	20.6	56.9	20.4	49.7	29.9	6.7	10.7	82.7	56.0	3.9	40.1
	Single-hop	**90.2**	0.5	9.3	70.7	19.8	9.5	61.6	13.5	24.9	55.0	6.2	38.8
	Summary	20.7	20.4	59.0	19.8	24.0	56.2	29.1	8.4	62.5	43.6	2.4	54.0
HybridRAG	Multi-hop	32.8	22.7	44.5	20.4	32.1	47.5	11.8	14.8	73.4	64.1	4.1	31.8
	Single-hop	83.7	1.5	14.8	80.0	13.7	6.4	**72.2**	15.1	12.7	**67.8**	5.7	26.5
	Summary	30.2	29.6	40.2	14.8	35.3	49.8	34.7	13.1	52.2	54.0	1.7	44.3
IterativeRAG	Multi-hop	21.8	21.9	56.3	17.1	37.3	45.6	12.4	13.9	73.8	**67.8**	4.3	27.9
	Single-hop	10.3	24.1	65.6	67.0	24.5	8.6	49.5	24.6	25.9	62.1	12.5	25.4
	Summary	21.2	34.2	44.6	16.3	38.2	45.6	30.5	10.8	58.8	**56.1**	3.8	40.1
Model: LLaMA-3.1-8B													
NaiveRAG	Multi-hop	7.9	9.3	82.7	10.9	20.4	68.8	10.7	21.3	68.1	37.9	8.1	54.0
	Single-hop	10.3	5.8	83.9	**69.2**	13.9	17.0	50.3	24.6	25.1	52.1	12.3	35.6
	Summary	12.0	47.8	40.2	1.4	65.4	33.2	22.8	27.8	49.3	30.1	14.5	55.4
GraphRAG	Multi-hop	9.7	13.6	76.7	9.5	12.8	77.7	7.4	12.2	80.4	33.6	6.3	60.1
	Single-hop	**84.4**	1.3	14.3	44.7	13.0	42.3	55.4	15.7	28.9	48.1	8.5	43.4
	Summary	11.4	37.5	51.1	2.5	67.8	29.7	19.7	26.5	53.8	31.8	9.7	58.5
HybridRAG	Multi-hop	12.1	14.9	73.0	15.6	21.5	62.9	13.1	19.0	67.9	39.9	6.1	54.0
	Single-hop	79.9	2.3	17.8	70.3	12.1	17.6	**63.8**	20.8	15.4	**55.7**	11.2	33.1
	Summary	13.9	48.6	37.5	2.5	68.6	29.0	24.2	27.0	48.8	33.9	10.4	55.7
IterativeRAG	Multi-hop	6.0	12.9	81.1	11.5	14.8	73.8	10.1	20.2	69.8	44.6	8.8	46.6
	Single-hop	12.3	32.9	54.8	62.3	16.1	21.6	48.6	25.7	25.7	47.4	20.5	32.1
	Summary	6.8	44.8	48.4	1.4	61.5	37.1	12.6	27.3	60.1	27.3	12.5	60.2

Table 16: Comprehensive LLM-as-a-Judge evaluation comparison between DeepSeek-V3 and LLaMA-3.1-8B across four datasets. Results report Accuracy (Cor), Incorrectness (Inc), and No-Answer rates (No-A).

Table 17: Multi-dimensional metrics for evaluating RAG generation quality.

Category	Metric	Focus	Description & Rationale
Answer Quality	LLM-as-a-Judge	Answer Correctness	LLM classifies answers as *correct*, *incorrect*, or *incomplete*, providing human-aligned judgment.
	Semantic F1	Reference Similarity	BERTScore-based token-level semantic similarity between prediction and ground truth, robust to paraphrase variations.
	Coverage	Completeness	Maximum cosine similarity between GT embedding and any prediction sentence, measuring information recall.
Grounding	Faithfulness (Hard)	Hallucination	Fraction of answer sentences with retrieval support above threshold ($\tau = 0.7$), detecting unsupported claims.
	Faithfulness (Soft)	Support Strength	Mean of max similarities between answer sentences and retrieval content, measuring grounding degree.

Grounding metrics verify whether the response is supported by the retrieved context, serving as a mechanism to identify hallucinations.

Answer Quality Metrics. Semantic F1 calculates the token-level semantic similarity between the generated response and the ground truth, derived from BERTScore. Let $\hat{y} = \{\hat{x}_1, \ldots, \hat{x}_m\}$ denote the predicted answer and $y = \{x_1, \ldots, x_n\}$ denote the reference answer. We first extract contextual embeddings utilizing a pre-trained language model, specifically `microsoft/deberta-xlarge-mnli`, and subsequently compute the precision (P_{BERT}), recall (R_{BERT}), and F1 score:

Dataset	Method	DeepSeek-V3					LLaMA-3.1-8B				
		Sem-F1	COV	Faith-H	Faith-S	LLM-Cor%	Sem-F1	COV	Faith-H	Faith-S	LLM-Cor%
MuSiQue	LLM-only	-	-	-	-	0.0	-	-	-	-	0.0
	NaiveRAG	0.503	0.362	0.143	0.486	26.4	0.249	0.161	0.194	0.540	8.6
	GraphRAG	0.510	0.386	0.114	0.439	30.3	0.374	0.262	0.185	0.494	18.7
	HybridRAG	0.613	0.472	-	-	38.6	0.406	0.284	0.178	0.582	20.3
	Iterative (Naive)	0.469	0.320	-	-	20.4	0.289	0.169	-	-	6.8
	Iterative (Graph)	-	0.400	-	-	29.2	-	0.271	-	-	16.1
QuALITY	LLM-only	-	-	-	-	0.0	-	-	-	-	0.0
	NaiveRAG	0.858	0.627	0.009	0.404	48.7	0.620	0.475	0.011	0.414	30.7
	GraphRAG	0.794	0.546	0.009	0.377	39.3	0.501	0.376	0.014	0.385	21.2
	HybridRAG	0.738	0.553	0.146	0.438	41.7	0.643	0.493	0.201	0.283	33.2
	Iterative (Naive)	0.724	0.506	-	-	35.8	0.583	0.435	-	-	28.4
	Iterative (Graph)	0.657	0.446	-	-	28.7	0.528	0.378	-	-	20.4
Legal	LLM-only	-	-	-	-	0.0	-	-	-	-	0.0
	NaiveRAG	0.568	0.469	0.145	0.537	32.2	0.587	0.473	0.164	0.531	25.8
	GraphRAG	0.530	0.443	0.094	0.510	29.3	0.536	0.445	0.146	0.508	25.0
	HybridRAG	0.617	0.520	0.326	0.589	36.1	0.622	0.512	0.349	0.627	31.1
	Iterative (Naive)	0.572	0.466	-	-	28.5	0.577	0.453	-	-	22.0
	Iterative (Graph)	0.534	0.439	-	-	26.0	-	0.444	-	-	21.0
Medical	LLM-only	-	-	-	-	0.0	-	-	-	-	0.0
	NaiveRAG	0.770	0.599	0.207	0.583	61.1	0.732	0.574	0.175	0.575	44.9
	GraphRAG	0.691	0.541	0.250	0.588	53.5	0.673	0.532	0.214	0.572	41.7
	HybridRAG	0.792	0.620	0.358	0.767	64.7	0.759	0.600	0.374	0.813	48.2
	Iterative (Naive)	0.826	0.595	-	-	62.7	0.802	0.582	-	-	43.6
	Iterative (Graph)	0.801	0.575	-	-	59.8	0.818	0.595	-	-	43.6

Table 18: Evaluation results on DeepSeek-V3 and LLaMA-3.1-8B across all datasets and RAG paradigms. Sem-F1: Semantic F1 (BERTScore-based), COV: Coverage, Faith-H/S: Faithfulness Hard/Soft, LLM-Cor%: LLM-as-a-Judge correct rate. "-" indicates metric not applicable or not computed.

$$P_{\text{BERT}} = \frac{1}{|\hat{y}|} \sum_{\hat{x}_i \in \hat{y}} \max_{x_j \in y} \cos(\mathbf{h}_{\hat{x}_i}, \mathbf{h}_{x_j}) \tag{10}$$

$$R_{\text{BERT}} = \frac{1}{|y|} \sum_{x_j \in y} \max_{\hat{x}_i \in \hat{y}} \cos(\mathbf{h}_{\hat{x}_i}, \mathbf{h}_{x_j}) \tag{11}$$

$$\text{Semantic F1} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{12}$$

Here, $\mathbf{h}$ represents the contextual embedding vector of a token, and $\cos(\cdot, \cdot)$ signifies cosine similarity. This metric exhibits robustness against synonym substitution and paraphrastic variations.

Coverage quantifies the extent to which the generated response encapsulates the information present in the gold standard. We utilize `SentenceTransformer` (`all-MiniLM-L6-v2`) to segment both the reference and the prediction into individual sentences and encode them into sentence-level embeddings. For each sentence s_i^{gt} in the ground truth, we compute its maximum similarity with respect to all sentences in the prediction:

$$\text{Coverage} = \frac{1}{|S^{gt}|} \sum_{s_i^{gt} \in S^{gt}} \max_{s_j^{pred} \in S^{pred}} \cos(\mathbf{e}_{s_i^{gt}}, \mathbf{e}_{s_j^{pred}}) \tag{13}$$

where S^{gt} and S^{pred} denote the sentence sets of the ground truth and the prediction, respectively, and $\mathbf{e}$ represents the sentence embedding. A higher coverage value indicates that the generated response has successfully captured a greater proportion of the critical information contained in the reference.

Faithfulness Metrics. The Faithfulness metric evaluates whether the generated response is faithful to the retrieved context, serving as a primary mechanism for detecting hallucinations. We segment the generated response into individual sentences and calculate the semantic support for each sentence against the retrieved content.

Faithfulness (Hard) utilizes a strict threshold to determine the proportion of sentences in the answer that are supported by the retrieval:

Algorithm 4 GraphRAG Retrieval & Generation Pipeline

Require: Question q, Knowledge Graph $G = (V, E)$, Entity Index $\mathcal{I}_E$, Token Budget B
Ensure: Generated Answer a
Hyperparameters: Entity Threshold $\tau_{\text{entity}} = 0.4$, PPR Threshold $\tau_{\text{ppr}} = 1e^{-5}$, Damping $\alpha = 0.85$

// Phase 1: Seed Entity Retrieval
$E_{\text{query}} \leftarrow \text{LLM}(\text{Prompt}_{\text{extract}}, q)$
if $E_{\text{query}} = \emptyset$ **then**
  $E_{\text{query}} \leftarrow \{q\}$ ▷ Fallback: use entire question
end if
$S \leftarrow \emptyset$ ▷ Initialize seed set mapping: $id \rightarrow score$
for each $e \in E_{\text{query}}$ **do**
  $v_e \leftarrow \text{Embed}(e)$
  $\mathcal{K} \leftarrow \text{FAISS_Search}(\mathcal{I}_E, v_e, k = 20)$
  for each $(id, \text{sim}) \in \mathcal{K}$ **do**
    if $\text{sim} > \tau_{\text{entity}}$ **then**
      $S[id] \leftarrow \max(S[id], \text{sim})$ ▷ Max pooling for duplicates
    end if
  end for
end for
$S \leftarrow \text{TopK}(S, k = 20)$

// Phase 2: PPR-Based Subgraph Expansion
$\mathbf{p} \leftarrow \text{Zeros}(|V|)$ ▷ Initialize personalization vector
$Z \leftarrow \sum_{(id,\text{sim}) \in S} \text{sim}$
for each $(id, \text{sim}) \in S$ **do**
  $\mathbf{p}[id] \leftarrow \text{sim}/Z$ ▷ Normalize to probability distribution
end for
$\boldsymbol{\pi} \leftarrow \text{PageRank}(G, \text{personalization} = \mathbf{p}, \alpha = \alpha, \text{iter} = 100)$
$V_{\text{sub}} \leftarrow \{v \mid \boldsymbol{\pi}[v] \geq \tau_{\text{ppr}}\}$
$V_{\text{sub}} \leftarrow \text{TopK}(V_{\text{sub}}, k = 100) \cup \text{Keys}(S)$ ▷ Keep top-100 expanded nodes + seeds

// Phase 3: Context Construction
$T \leftarrow \{(u, r, v) \mid u, v \in V_{\text{sub}}, (u, v) \in E\}$ ▷ Extract triplets from induced subgraph
$\mathcal{C} \leftarrow \emptyset$
Sort T by $\max(\text{sim}(u), \text{sim}(v))$ descending ▷ Prioritize relevance
$ctx \leftarrow$ "", count $\leftarrow 0$
for each $(u, r, v) \in T$ **do**
  sents $\leftarrow \text{TripletSourceMap}[(u, r, v)]$
  for each $s \in$ sents **do**
    if $\text{count} + \text{Len}(s) > B$ **then break**
    end if
    $ctx \leftarrow ctx \oplus s$
    $\text{count} \leftarrow \text{count} + \text{Len}(s)$
  end for
end for

// Phase 4: Answer Generation
$a \leftarrow \text{LLM}(\text{Prompt}_{\text{RAG}}, ctx, q)$
return a

$$\text{Faith}_{\text{hard}} = \frac{1}{|S^{ans}|} \sum_{s_i \in S^{ans}} \mathbb{1}\left[\max_{c_j \in C} \cos(\mathbf{e}_{s_i}, \mathbf{e}_{c_j}) \geq \tau\right] \tag{14}$$

where S^{ans} denotes the set of answer sentences, C denotes the set of retrieved context sentences, $\tau = 0.7$ serves as the similarity threshold, and $\mathbb{1}[\cdot]$ is the indicator function. This metric strictly quantifies the fraction of the response that possesses explicit grounding within the retrieved results.

Faithfulness (Soft) employs a continuous calculation to measure the average support strength between the answer and the retrieved content:

$$\text{Faith}_{\text{soft}} = \frac{1}{|S^{ans}|} \sum_{s_i \in S^{ans}} \max_{c_j \in C} \cos(\mathbf{e}_{s_i}, \mathbf{e}_{c_j}) \tag{15}$$

The Soft version provides a more granular measure of grounding, reflecting partial support even when the strict threshold is not met.

It is important to note that Faithfulness metrics are not computed for IterativeRAG. We exclude it from this specific evaluation because its multi-turn loop accumulates context across rounds, so an answer sentence cannot be attributed to a single retrieval result without bias.

LLM-as-a-Judge. Complementing automated metrics, we employ the LLM-as-a-Judge methodology to conduct human-aligned correctness evaluation. As illustrated in Figure 17, we devise a structured evaluation prompt instructing the evaluator model (GPT-4o) to classify generated responses into three mutually exclusive categories. (1) Correct: The response is logically accurate and encapsulates the core information of the ground truth. (2) Incorrect: The response contains erroneous information that contradicts the ground truth. (3) Incomplete: The response is partially correct yet lacks critical details, or the model refuses to generate an answer.

This evaluation paradigm mitigates the limitations of similarity-based metrics, which often fail to capture logical inconsistencies, thereby providing a quality assessment that more closely aligns with human judgment.

Evaluation Results. Table 18 presents the comprehensive evaluation results for DeepSeek-V3 and LLaMA-3.1-8B across the four constituent datasets. The key findings are summarized as follows:

Overall Performance: HybridRAG achieves optimal or near-optimal performance across the majority of datasets, particularly excelling in Semantic F1 and Coverage metrics. For instance, on the Medical dataset, DeepSeek-V3 combined with HybridRAG attains an LLM accuracy of 64.7%, surpassing both NaiveRAG (61.1%) and GraphRAG (53.5%).

Model Disparity: DeepSeek-V3 significantly outperforms LLaMA-3.1-8B. Taking the MuSiQue

dataset as an example, DeepSeek-V3 with HybridRAG achieves an LLM accuracy of 38.6%, whereas LLaMA-3.1-8B reaches only 20.3%, a substantial performance gap of 18.3 percentage points.

Faithfulness Analysis: Faithfulness scores for NaiveRAG and GraphRAG are generally low (mostly below 0.25), indicating that even with retrieval augmentation, models continue to generate content unsupported by the retrieved context. Conversely, the Medical dataset exhibits relatively higher faithfulness (GraphRAG reaches 0.250 Hard / 0.588 Soft), potentially attributable to the medical domain's strict reliance on retrieved factual evidence for answer formulation. HybridRAG is the most grounded of the three, reaching 0.358 Hard / 0.767 Soft on Medical, which is consistent with its fused context covering more of the evidence an answer draws on.

Table 16 further provides a breakdown of LLM evaluation results by query type. Key observations include:

Factual Dominance: All methods yield their best performance on single-hop queries. GraphRAG achieves accuracies of 90.2% (DeepSeek) and 84.4% (LLaMA-3.1-8B) on MuSiQue's single-hop questions, significantly outperforming other query types. This aligns with GraphRAG's entity-based retrieval mechanism, where single-entity queries facilitate the precise localization of relevant information.

Reasoning Challenge: Multi-hop queries prove universally challenging across all methods, with accuracy generally remaining below 35%. The Legal dataset is particularly demanding, where the best-performing method (IterativeRAG) attains only 12.4% accuracy on multi-hop tasks.

Summary Dilemma: Summary-type queries exhibit a high rate of incorrect responses (exceeding 30% for most methods on QuALITY), suggesting a model tendency to generate summaries that are either over-generalized or deviate from the source text.

Statistical Robustness. All reported numbers are computed on the full set of 7,727 queries rather than on a random sample, and generation is run once per configuration under a fixed decoding temperature, so the reported values are exact statistics of the benchmark rather than point estimates of a sampling procedure. To quantify how much of the observed paradigm ordering would survive a different draw of queries, we additionally perform query-level paired bootstrap resampling with 10,000 iterations (Koehn, 2004; Dror et al., 2018). Table 19 reports the resulting 95% confidence intervals on DeepSeek-V3. The intervals of all four retrieval paradigms are mutually non-overlapping, and the complete paradigm ranking is reproduced in 9,995 of the 10,000 resamples (99.95%), indicating that the effectiveness ordering is not an artifact of the particular query set.

Paradigm	Correct rate	95% CI
HybridRAG	45.04	[43.91, 46.16]
NaiveRAG	39.32	[38.23, 40.40]
GraphRAG	37.23	[36.15, 38.33]
IterativeRAG	34.50	[33.45, 35.55]
LLM-only	0.00	[0.00, 0.00]

Table 19: **Bootstrap confidence intervals on DeepSeek-V3.** Correct rate (%) aggregated over all 7,727 queries, with 95% confidence intervals from 10,000 query-level paired bootstrap resamples. Ranking consistency across resamples: $9{,}995/10{,}000 = 99.95\%$.

C.2 Judge Reliability

Because LLM-as-a-Judge outputs serve both as an evaluation metric and as the basis for routing labels, we validate the judge along three independent axes. These checks concern the evaluation of *generated answers*, and are distinct from the human verification of *generated queries* reported in Appendix A.2.

Agreement with Human Annotation. On a stratified sample of $N = 100$ RAG outputs, judge labels reach Cohen's $\kappa = 0.803$ against independent author annotation, above the conventional threshold for substantial agreement ($\kappa \geq 0.61$).

Cross-Vendor Agreement. To test whether the labels depend on the identity of the judge model, we re-label a stratified pool of $N = 200$ RAG outputs, spanning all four RAG paradigms, both backbone LLMs, and all four corpora, with Claude-Sonnet-4.5. The two judges reach 90.5% raw agreement and Cohen's $\kappa = 0.847$. Per-paradigm agreement is uniformly high (NaiveRAG 92.0%, GraphRAG 86.0%, HybridRAG 92.0%, IterativeRAG 92.0%), and cross-category swaps between *correct* and *incorrect* account for only 4 of the 200 cases (Table 20).

Default judge	*correct*	*incorrect*	*incomplete*
correct (60)	59	0	1
incorrect (40)	4	32	4
incomplete (100)	8	2	90

Table 20: **Cross-vendor judge confusion matrix** ($N = 200$). Rows: default judge; columns: Claude-Sonnet-4.5.

Rubric Sensitivity. On the same pool, we re-judge under two alternative rubrics, `strict`, which requires near-exact key-fact match, and `lenient`, which rewards substantive coverage. Agreement with the default rubric is 86.0% ($\kappa = 0.768$) for `strict` and 83.5% ($\kappa = 0.742$) for `lenient`, so individual labels remain stable under rephrasing. Absolute correct rates, by contrast, move by roughly ten percentage points with rubric strictness, and on a pool of this size the ordering among the leading paradigms is not stable either, with default / strict / lenient giving NaiveRAG $34.0/24.0/46.0$, GraphRAG $24.0/12.0/40.0$, HybridRAG $32.0/24.0/50.0$ and IterativeRAG $30.0/26.0/42.0$. What holds under all three rubrics is that GraphRAG ranks last and that the remaining three sit within a few points of one another. Taken together with the human and cross-vendor agreement above, the checks indicate that judge labels are reliable at the instance level, which is what the routing labels depend on, while paradigm-level separation on a sample of this size should be read as indicative. The ordering reported in Section 3.4 rests instead on the full benchmark, where the bootstrap intervals of Table 19 over all 7,727 queries are mutually non-overlapping.

C.3 Cost Calculation

Cost Components. We decompose the computational overhead of the RAG system into two primary phases (Table 21): the Retrieval phase and the Generation phase. The total cost is formally defined as:

$$C_{\text{total}} = C_{\text{retrieval}} + C_{\text{generation}} \tag{16}$$

where the cost for each phase comprises both input and output token consumption:

$$C_{\text{retrieval}} = T_{\text{ret}}^{\text{in}} + T_{\text{ret}}^{\text{out}} \tag{17}$$

$$C_{\text{generation}} = T_{\text{gen}}^{\text{in}} + T_{\text{gen}}^{\text{out}} \tag{18}$$

In the generation phase, the input token volume is predominantly governed by the aggregate context length:

$$T_{\text{gen}}^{\text{in}} \approx N \times (L_{\text{prompt}} + L_{\text{context}} + L_{\text{query}}) \tag{19}$$

where N denotes the total number of queries, L_{prompt} represents the fixed length of the system prompt, L_{context} is the average length of the retrieved context (denoted as *Avg-Ctx* in the table), and L_{query} is the query length.
For GraphRAG and HybridRAG, the retrieval cost incorporates a one-time graph construction overhead, which is amortized over the query set:

$$C_{\text{retrieval}}^{\text{graph}} = \underbrace{C_{\text{construction}}}_{\text{one-time, amortized}} + \underbrace{C_{\text{entity_extraction}}}_{\text{per-query}} \tag{20}$$

The construction cost encompasses the input tokens required for processing the raw corpus via the LLM (T_{corpus}) and the resulting output tokens for the extracted triplets (T_{triplets}).

Method Comparison. Table 21 reveals substantial disparities in computational cost across the five distinct methodologies:
LLM-only: It incurs minimal overhead (0.04–0.10M tokens), as it bypasses retrieval and context injection, consuming tokens solely for the prompt, query input, and response generation.
NaiveRAG: Its cost footprint is predominantly driven by the generation phase ($T_{\text{gen}}^{\text{in}}$ accounts for $> 99\%$), necessitated by the inclusion of extensive retrieved chunks within the context window. For instance, on the Medical dataset, the average context length reaches 50,504 tokens, resulting in a total expenditure of 95.86M tokens.
GraphRAG: It generally exhibits lower generation costs compared to NaiveRAG, as graph-based retrieval yields more precise and concise contexts. On MuSiQue, GraphRAG records an Avg-Ctx of 8,472 (vs. 13,139 for NaiveRAG), translating to a total cost of 31.14M (vs. 44.22M), a reduction of approximately 30%. However, the Legal dataset presents an exception; here, GraphRAG's Avg-Ctx surges to 179,728. This anomaly arises from the dense entity interconnectivity characteristic of legal documents, where graph traversal retrieves a voluminous amount of associated content.
HybridRAG: It incurs the highest computational burden, as it utilizes results from both vector and graph retrieval, resulting in a context length approximating the sum of both. On the Legal dataset, it peaks at 293.10M tokens, marking the maximum consumption across all evaluated methods.

Dataset	Method	N	Avg-Ctx	Ret-In	Ret-Out	Gen-In	Gen-Out	Total
MuSiQue	LLM-only	3,356	0	0	0	0.07	0.03	0.10
	NaiveRAG	3,356	13,139	0	0	44.17	0.05	44.22
	GraphRAG	3,356	8,472	2.35	0.24	28.51	0.04	31.14
	HybridRAG	3,356	21,602	2.35	0.24	72.57	0.06	75.22
	IterativeRAG	3,356	6,364	43.24	0.72	21.44	0.05	65.46
QuALITY	LLM-only	1,198	0	0	0	0.03	0.00	0.04
	NaiveRAG	1,198	49,444	0	0	59.27	0.04	59.31
	GraphRAG	1,198	48,502	1.56	0.07	58.14	0.04	59.81
	HybridRAG	1,198	97,789	1.56	0.07	117.19	0.03	118.85
	IterativeRAG	1,198	6,871	16.69	0.25	8.27	0.03	25.24
Legal	LLM-only	1,277	0	0	0	0.05	0.01	0.06
	NaiveRAG	1,277	46,273	0	0	59.14	0.05	59.19
	GraphRAG	1,277	179,728	4.82	0.13	229.56	0.04	234.55
	HybridRAG	1,277	225,571	4.82	0.13	288.10	0.05	293.10
	IterativeRAG	1,277	6,460	16.81	0.29	8.30	0.05	25.45
Medical	LLM-only	1,896	0	0	0	0.03	0.06	0.09
	NaiveRAG	1,896	50,504	0	0	95.78	0.08	95.86
	GraphRAG	1,896	37,513	0.25	0.15	71.15	0.07	71.63
	HybridRAG	1,896	73,628	0.25	0.15	139.63	0.08	140.11
	IterativeRAG	1,896	2,231	8.67	0.26	4.26	0.08	13.27

Table 21: Token consumption breakdown for retrieval and generation across all datasets and methods. All token counts are in millions (M) except Avg-Ctx (average context tokens per question). Ret: Retrieval, Gen: Generation, In: Input, Out: Output.

IterativeRAG: It exhibits a distinct cost structure characterized by high retrieval overhead (due to multi-turn LLM invocations for judgment and subquery generation) but low generation cost (owing to the refined conciseness of the context). On the Medical dataset, despite a substantial $T_{\text{ret}}^{\text{in}}$ of 8.67M, the Avg-Ctx remains merely 2,231, yielding a total cost of 13.27M, the lowest among all RAG paradigms.

Dataset Variation. The cost variations across datasets are predominantly governed by document length and corpus scale:

The Legal dataset: It incurs significantly higher costs compared to other benchmarks. Specifically, GraphRAG's total expenditure on Legal (234.55M) is 7.5 times that on MuSiQue (31.14M). This disparity stems from the extensive length and complex entity interrelations inherent in legal documents, which result in: (1) elevated graph construction overhead; and (2) substantially longer contexts yielded by graph traversal (with Avg-Ctx reaching 179,728).

The Narrative dataset: It's costs of NaiveRAG and GraphRAG are comparable (59.31M vs. 59.81M), indicating that graph-based retrieval fails to effectively reduce context length in this setting. This is attributable to QuALITY's long-document characteristic, where each question corresponds to a complete article, resulting in high inter-chunk correlation.

The Medical dataset: Its IterativeRAG demonstrates superior cost-efficiency (13.27M), amounting to merely 14% of the cost incurred by NaiveRAG (95.86M). Medical QA typically involves explicit information needs, allowing iterative retrieval to rapidly localize critical content.

Cost-Performance Trade-off. Synthesizing the cost profiles in Table 21 with the performance metrics in Table 18, we analyze the cost-performance trade-offs:

HybridRAG: High Cost, High Performance. On MuSiQue, HybridRAG attains an LLM accuracy of 38.6% at a cost of 75.22M tokens. Compared to NaiveRAG (44.22M, 26.4%), this represents a 70% cost increase yielding a 46% performance gain. Conversely, on **Medical**, HybridRAG (140.11M, 64.7%) incurs a 46% cost hike over NaiveRAG (95.86M, 61.1%) for a mere 6% performance improvement, indicating diminishing marginal returns.

GraphRAG: Dataset-Dependent Cost-Efficiency. On MuSiQue, GraphRAG (31.14M, 30.3%) delivers superior performance at a lower cost than NaiveRAG, emerging as the optimal choice. However, on QuALITY, GraphRAG (59.81M, 39.3%)

offers no advantage, incurring costs comparable to NaiveRAG while yielding inferior performance (48.7%).

IterativeRAG: Low Cost, Variable Performance. On Medical, IterativeRAG (Naive-base) achieves the highest cost-effectiveness, reaching 62.7% accuracy at a minimal cost of 13.27M. Yet, on MuSiQue, the same approach yields only 20.4% accuracy, underperforming other more resource-intensive methods.

Practical Recommendations: (1) For domains with explicit entity relations (e.g., Medical), IterativeRAG offers the best cost-performance ratio; (2) For complex QA requiring the synthesis of multi-source information, HybridRAG delivers optimal performance despite its high cost; and (3) For long-document comprehension tasks (e.g., QuALITY), NaiveRAG remains the most straightforward and effective solution.

C.4 Cost-Aware Label Construction

The main experiments adopt the quality-first label of Section 3.5, which selects the highest-Semantic-F1 paradigm among those judged correct and evaluates cost as a downstream deployment metric. The same formulation also admits cost-aware labels through $\arg\max_\pi(\mathrm{F1}_\pi - \lambda \widetilde{c}_\pi)$, where $\widetilde{c}_\pi$ is the normalized token cost of paradigm π; queries for which no paradigm is correct retain the *cannot answer* label regardless of λ.

Sweeping λ on DeepSeek-V3 under Q+C features and 5-fold cross-validation shows that the two objectives are far from symmetric in their sensitivity. Relative to $\lambda = 0$, any weight $\lambda \geq 0.25$ relabels 2,242 of the 7,727 queries (29%) and reduces total token consumption by 41%, to $0.24\times$ the cost of Always HybridRAG, while correct rate decreases by only 0.6 percentage points. The relabeling is dominated by a shift toward the cheapest correct paradigm: the HybridRAG label count falls from 1,006 to 162 while IterativeRAG rises from 743 to 2,096. Because per-query cost differentials between paradigms are large and well separated, the induced labeling is stable across the tested range of λ rather than varying continuously with it.

This asymmetry is a property of the benchmark rather than of any particular router: when a query is answerable by several paradigms, the cheapest correct one is usually almost as accurate as the most expensive one. It also identifies multi-objective router design, including continuous trade-off curves and learned trade-off weights, as a direction that RAGRouter-Bench is able to measure.

C.5 Case Studies

Comparative Analysis. Table 22 presents qualitative analysis of representative cases, comparing paradigm performance on multi-hop reasoning and cross-document summarization tasks.

Error Analysis. Drawing upon the aforementioned case studies and the comprehensive evaluation results, we categorize the primary failure modes as follows:

Retrieval Imprecision: The failure of NaiveRAG and HybridRAG in Case 1 stems from the fact that while the retrieved content possessed thematic relevance (cervical cancer surgery), it failed to precisely hit the pivotal term "trachelectomy." This error is particularly prevalent in specialized domains (e.g., Medical, Legal), where the semantic similarity of domain-specific terminology may be lower than that of generic descriptions. GraphRAG, through its entity matching mechanism, demonstrates superior precision in localizing such specialized terms.

Context Overload: When retrieval yields a high volume of relevant yet redundant content, the LLM is prone to overlooking critical information. In Case 1, HybridRAG's average context length (Avg-Ctx) reached 73,628 tokens. Such an excessively long context can cause the model to become "lost in the middle," resulting in performance inferior to that of GraphRAG, which utilizes a more concise context (Avg-Ctx: 37,513).

Hallucination: In Case 2, GraphRAG generated concepts absent from the source documents (e.g., "transphasia"). This indicates that when the alignment between retrieved content and the query is poor, the model may resort to its internal parametric knowledge to "complete" the answer, leading to fabrication. This issue is particularly pronounced in Summary-type queries, as summarization necessitates cross-document synthesis which fragmented retrieval results often fail to fully cover.

Retrieval Mismatch: In Case 2, HybridRAG output "James I", an answer entirely unrelated to spacecraft, suggesting that the retrieval module returned completely irrelevant document fragments. Such severe mismatches likely stem from a semantic gap between the query and the corpus, or from errors in entity extraction within the graph retrieval process.

Reasoning Chain Failure: For questions necessitating multi-step reasoning (e.g., the multi-hop

Case ID	Query & Gold Standard	Paradigm Comparison	Key Analysis
Case 1 *Medical 1244* (Multi-hop)	**Q:** What are the surgical options for early cervical cancer and how do they relate to fertility preservation? **Gold:** Cone biopsy and trachelectomy are surgical options for early-stage disease, with trachelectomy being a fertility-sparing procedure.	**GraphRAG (Correct):** Explicitly states that "fertility-sparing surgical options include cone biopsy or radical trachelectomy," accurately capturing both procedures. **NaiveRAG (Incomplete):** Mentions cone biopsy but fails to explicitly cite "trachelectomy," offering generic "hysterectomy types." **HybridRAG (Incomplete):** Similarly omits "trachelectomy," referencing only distinct hysterectomy types. **IterativeRAG (Correct):** Successfully localizes "cone biopsy, trachelectomy" through iterative retrieval, fully covering the gold answer.	This case demonstrates that for multi-hop queries necessitating precise medical terminology, GraphRAG's entity-oriented retrieval and IterativeRAG's multi-turn refinement are effective. In contrast, semantic-based methods (Naive/Hybrid) lack the required precision.
Case 2 *QuALITY 1034* (Summary)	**Q:** Based on the narrative, what are the common types of extreme adversity faced by spacecraft? What are the crew's survival strategies? **Gold:** Scenarios include physical trauma (crash/flip), atmospheric entry, and combat. Reactions involve assessing reparability or risking hyperdrive escape.	**GraphRAG (Incorrect):** Hallucinates concepts like "transphasia" and "space cafard" absent from the source text. **NaiveRAG (Incorrect):** Lists generic tropes like "sabotage, alien attacks" without addressing specific scenarios. **HybridRAG (Incorrect):** Erroneously outputs "James I," suggesting irrelevant retrieval. **IterativeRAG (Incomplete):** Closest result, citing "mechanical failures" and "damaged hulls," capturing the thematic direction but missing details.	This highlights the challenge of Summary-type queries requiring cross-document synthesis. Even IterativeRAG only achieves Incomplete status, indicating significant room for improvement in long-document summarization tasks.

Table 22: Qualitative analysis of two representative cases.

query in Case 1), even if partially relevant content is retrieved, the model may fail to correctly link the information. IterativeRAG, via its multi-turn "Retrieve-Generate-Evaluate" loop, effectively patches the reasoning chain step-by-step, thereby exhibiting relatively stable performance on complex queries.

D Router Evaluation Details

D.1 Router Architecture Details

Table 23 summarizes the architecture and key hyperparameters of all nine routers evaluated in this work.

Router	Category	Key Configuration
LR	Traditional ML	L2 regularization
SVM	Traditional ML	RBF kernel, probability=True
XGBoost	Traditional ML	100 estimators, max_depth=6
MLP	Traditional ML	Hidden layers: 64, 32
DNN	Neural	3-layer MLP (256→128→C) with BN
TwoTower	Neural	Query tower + paradigm embedding
Adaptive-RAG	Neural	LM classifier (Q+C adapted)
MBA-RAG	Neural	Multi-armed bandit with reward MLP
RouterDC	Neural	Dual contrastive learning

Table 23: Router architecture summary. C denotes the number of paradigm classes for the given backbone. All neural routers use dimension-aligned feature processing, in which query text is encoded via a pretrained language model and projected to 64 dimensions, corpus indicators are discretized into bins and embedded into 64 dimensions, and the resulting 128-dimensional vector serves as input.

D.2 Feature Processing and Training Protocol

For traditional ML routers, query features are encoded as a 3-dimensional one-hot vector of query type (factual, reasoning, summary), and corpus features as 6 dual-view indicators (LCC Ratio, Density, Clustering Coefficient, Intrinsic Dimension, Hubness, Average Distance). The Q+C input is their concatenation (9 dimensions).

For neural routers, query text is encoded by a pretrained language model (BERT or BGE) and projected to 64 dimensions via a linear layer. Corpus indicators are discretized into 10 bins per feature and each bin is mapped to a learnable embedding, which are concatenated and projected to 64 dimensions. This ensures balanced dimensionality between the two signal sources before concatenation

Table 24: **Complete router evaluation across all architectures, feature configurations, and datasets (DeepSeek-V3 backbone).** Results are reported for both BERT and BGE encoders under query-only (Q), corpus-only (C), and joint (Q+C) feature settings. R.A: routing accuracy (%), F1: macro F1 (%), MRR: mean reciprocal rank, C.R: correct rate (%). The main text reports BERT results; BGE results serve as a robustness check.

Method	Feat.	Avg				MuSi.				QuAL.				Legal				Med.			
		R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R
Model: DeepSeek-V3																					
Random	–	20.1	18.9	–	39.1	20.0	–	–	28.4	20.0	–	–	42.8	20.1	–	–	31.6	20.2	–	–	60.6
Best-fixed	–	20.9	6.9	–	39.3	15.5	–	–	26.4	30.1	–	–	48.8	18.7	–	–	32.2	26.3	–	–	61.1
LR	Q	43.9	17.2	0.6374	39.3	48.4	15.2	0.6769	26.4	55.1	25.1	0.7152	48.9	53.1	22.4	0.6974	32.2	22.6	12.1	0.4775	61.1
	C	41.9	11.6	0.6236	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5037	61.1
	Q+C	46.8	21.0	0.6502	43.5	55.3	24.3	0.7106	35.8	55.1	25.1	0.7104	48.9	52.3	21.9	0.6945	32.8	22.6	12.1	0.4749	61.1
SVM	Q	43.9	17.2	0.6374	39.3	48.4	15.2	0.6769	26.4	55.1	25.1	0.7152	48.9	53.1	22.4	0.6974	32.2	22.6	12.1	0.4775	61.1
	C	41.9	11.6	0.6218	39.3	48.4	13.0	0.6638	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5040	61.1
	Q+C	47.6	20.9	0.6598	43.2	55.3	24.3	0.7135	35.8	55.1	25.1	0.7174	48.9	53.1	22.4	0.6993	32.2	25.4	11.2	0.5016	60.2
XGBoost	Q	43.9	17.2	0.6374	39.3	48.4	15.2	0.6769	26.4	55.1	25.1	0.7152	48.9	53.1	22.4	0.6974	32.2	22.6	12.1	0.4775	61.1
	C	41.9	11.6	0.6236	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5037	61.1
	Q+C	45.8	20.6	0.6595	42.9	52.8	24.1	0.7142	35.4	51.5	24.8	0.7181	48.7	52.8	21.9	0.6999	32.0	25.0	10.8	0.4985	59.9
MLP	Q	43.7	17.7	0.6305	39.9	49.8	17.2	0.6764	28.0	52.0	23.6	0.6904	48.5	53.0	22.4	0.6932	32.9	21.4	11.5	0.4693	60.5
	C	41.9	11.6	0.6196	39.3	48.4	13.0	0.6638	26.4	39.9	11.4	0.6286	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.4971	61.1
	Q+C	46.8	19.9	0.6518	42.9	54.8	23.9	0.7066	35.3	54.6	24.7	0.7099	48.5	51.4	19.7	0.6902	31.9	24.5	10.0	0.4917	60.3
DNN (BERT)	Q	44.2	28.8	0.6361	43.1	51.6	34.5	0.6912	35.2	45.7	25.5	0.6523	46.9	51.2	25.8	0.6767	33.7	25.3	22.7	0.5007	61.2
	C	41.9	11.6	0.6238	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6320	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5048	61.1
	Q+C	45.5	27.8	0.6452	43.4	53.4	33.6	0.6994	35.5	48.3	24.8	0.6729	48.0	52.8	26.6	0.6915	33.9	24.7	20.1	0.4997	60.9
TwoTower (BERT)	Q	44.0	25.9	0.6370	43.1	50.5	29.1	0.6833	33.9	44.8	23.0	0.6521	48.1	52.9	26.8	0.6949	34.0	25.6	21.5	0.5056	62.5
	C	41.9	11.6	0.6234	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5028	61.1
	Q+C	45.0	25.4	0.6424	42.8	51.4	28.3	0.6903	33.7	49.1	24.5	0.6788	48.2	53.3	26.1	0.6951	33.6	25.2	20.5	0.4992	61.8
Adaptive-RAG (BERT)	Q	45.0	26.6	0.6432	43.1	51.5	30.5	0.6927	33.8	47.4	24.0	0.6707	48.1	53.3	26.2	0.6951	34.1	26.1	21.7	0.5032	62.6
	C	41.9	11.6	0.6237	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5040	61.1
	Q+C	46.2	23.2	0.6523	42.9	53.0	26.7	0.7006	34.3	51.9	24.8	0.6975	49.0	53.1	25.0	0.6981	33.4	25.9	14.8	0.5070	60.6
MBA-RAG (BERT)	Q	44.1	24.7	0.6290	42.6	50.8	27.5	0.6766	33.5	45.6	22.9	0.6440	46.8	53.0	26.8	0.6886	34.1	25.4	19.3	0.4950	61.9
	C	41.9	11.6	0.6194	39.3	48.4	13.0	0.6635	26.4	39.9	11.4	0.6299	48.9	49.5	13.2	0.6744	32.2	26.3	8.3	0.4976	61.1
	Q+C	45.2	21.5	0.6405	42.7	51.5	25.1	0.6848	33.7	50.2	23.4	0.6834	48.8	52.5	22.6	0.6867	32.6	25.9	13.0	0.5034	61.3
RouterDC (BERT)	Q	43.3	27.3	0.6309	42.7	50.9	33.4	0.6845	34.9	42.9	24.1	0.6364	47.1	51.3	25.1	0.6816	33.1	24.8	20.0	0.4984	60.1
	C	41.5	11.6	0.6218	39.0	48.4	13.0	0.6667	26.4	39.9	11.4	0.6322	48.9	49.5	13.2	0.6757	32.2	25.0	8.0	0.4988	59.6
	Q+C	46.0	28.8	0.6484	43.7	52.6	35.0	0.6961	35.5	51.6	27.9	0.6926	48.6	53.2	26.8	0.6951	33.8	25.7	19.7	0.5046	61.7
DNN (BGE)	Q	42.2	28.9	0.6238	42.9	50.1	34.0	0.6788	34.0	40.9	23.3	0.6200	47.1	49.3	28.7	0.6693	33.4	24.2	23.6	0.4968	62.5
	C	41.9	11.6	0.6233	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5025	61.1
	Q+C	43.9	29.8	0.6359	43.4	51.0	35.1	0.6864	35.0	44.8	24.9	0.6424	46.9	51.0	27.5	0.6820	33.0	26.1	24.8	0.5101	62.9
TwoTower (BGE)	Q	42.2	24.2	0.6216	41.7	49.4	27.4	0.6732	31.9	44.0	22.8	0.6425	48.3	51.0	25.7	0.6770	32.9	22.3	18.5	0.4794	60.8
	C	41.9	11.6	0.6235	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5034	61.1
	Q+C	43.3	25.4	0.6299	42.3	49.7	29.0	0.6763	32.3	46.0	23.5	0.6559	48.3	52.5	25.6	0.6887	33.2	24.0	20.1	0.4911	62.2
Adaptive-RAG (BGE)	Q	42.2	27.4	0.6222	42.8	47.8	31.3	0.6645	33.4	43.5	25.1	0.6344	47.2	51.4	27.0	0.6798	33.8	25.3	22.4	0.5005	62.6
	C	41.9	11.6	0.6236	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6321	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5037	61.1
	Q+C	43.9	26.2	0.6359	42.4	50.0	29.6	0.6783	33.2	47.5	25.0	0.6670	48.9	51.8	25.1	0.6877	32.6	25.6	21.9	0.5053	61.2
MBA-RAG (BGE)	Q	39.7	25.7	0.5984	41.5	47.8	30.8	0.6546	31.9	38.5	22.2	0.5922	46.8	44.6	23.3	0.6273	31.2	23.0	20.4	0.4836	62.1
	C	41.9	11.6	0.6221	39.3	48.4	13.0	0.6646	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6762	32.2	26.3	8.3	0.5036	61.1
	Q+C	43.3	25.0	0.6215	41.6	48.5	26.7	0.6566	30.7	47.6	23.6	0.6568	48.7	50.1	27.5	0.6678	33.0	26.8	21.4	0.5060	62.2
RouterDC (BGE)	Q	41.2	28.0	0.6148	42.6	48.8	33.8	0.6687	33.6	39.3	21.8	0.6092	46.3	49.0	26.2	0.6643	33.5	23.6	22.7	0.4887	62.3
	C	41.9	11.6	0.6242	39.3	48.4	13.0	0.6680	26.4	39.9	11.4	0.6323	48.9	49.5	13.2	0.6756	32.2	26.3	8.3	0.5063	61.1
	Q+C	43.8	28.8	0.6323	42.8	50.2	34.6	0.6766	34.2	44.8	23.8	0.6454	47.1	52.9	27.5	0.6936	32.7	25.9	22.7	0.5044	62.3
Oracle	–	100.0	100.0	1.0000	60.8	100.0	–	–	51.6	100.0	–	–	59.9	100.0	–	–	50.5	100.0	–	–	84.7

into a 128-dimensional input vector.

Training protocol. All routers are trained and evaluated with 5-fold stratified cross-validation over the 7,727 query–corpus pairs, stratifying on the paradigm label so that every fold preserves the class distribution, including the *cannot answer* class. Reported numbers are averages over the five folds. Traditional ML routers use the regularization settings listed in Table 23; neural routers are trained to convergence with early stopping on the held-out fold. The same protocol is used for the λ sweep of Appendix C.4.

D.3 Evaluation Metrics

We report four complementary metrics. **Routing Accuracy (R.A)** measures the proportion of queries for which the router selects the optimal paradigm matching the ground-truth label. **Macro F1** computes the unweighted average of per-class F1 scores over the label space, capturing balanced performance across imbalanced class distributions. The label space comprises the paradigms that are optimal for at least one query under a given backbone, together with the *cannot answer* class, and is therefore backbone-dependent; this is why the Random baseline differs between the two panels of Table 3. **Mean Reciprocal Rank (MRR)** evaluates the ranking quality of predicted paradigm proba-

Table 25: **Complete router evaluation across all architectures, feature configurations, and datasets (LLaMA-3.1-8B backbone).** Results are reported for both BERT and BGE encoders under query-only (Q), corpus-only (C), and joint (Q+C) feature settings. R.A: routing accuracy (%), F1: macro F1 (%), MRR: mean reciprocal rank, C.R: correct rate (%). The main text reports BERT results; BGE results serve as a robustness check.

Method	Feat.	Avg				MuSi.				QuAL.				Legal				Med.			
		R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R	R.A	F1	MRR	C.R
Model: LLaMA-3.1-8B																					
Random	–	24.9	21.9	–	26.6	24.9	–	–	16.6	25.2	–	–	26.6	24.9	–	–	26.6	24.9	–	–	44.1
Best-fixed	–	14.4	6.3	–	25.8	12.8	–	–	18.7	9.3	–	–	21.2	13.5	–	–	25.0	21.1	–	–	41.7
LR	Q	61.1	30.0	0.7549	25.8	78.8	36.2	0.8605	18.7	59.2	28.3	0.7527	21.3	60.0	29.6	0.7485	25.1	31.8	20.4	0.5731	41.7
	C	59.4	18.6	0.7497	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7597	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5916	41.7
	Q+C	63.5	31.6	0.7684	27.7	78.8	36.2	0.8661	18.7	69.2	35.8	0.8036	30.6	59.9	29.5	0.7487	24.2	35.3	22.1	0.5859	44.0
SVM	Q	61.1	30.0	0.7531	25.8	78.8	36.2	0.8365	18.7	59.2	28.3	0.7867	21.3	60.0	29.6	0.7481	25.1	31.8	20.4	0.5866	41.7
	C	59.4	18.6	0.7495	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7587	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5916	41.7
	Q+C	63.1	30.0	0.7687	27.2	78.8	36.2	0.8691	18.7	69.2	35.8	0.7907	30.6	59.4	28.8	0.7431	24.1	33.8	16.1	0.5934	42.4
XGBoost	Q	61.1	30.0	0.7549	25.8	78.8	36.2	0.8605	18.7	59.2	28.3	0.7527	21.3	60.0	29.6	0.7485	25.1	31.8	20.4	0.5731	41.7
	C	59.4	18.3	0.7497	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7597	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5916	41.7
	Q+C	62.5	29.0	0.7700	26.7	78.5	35.9	0.8691	18.5	67.1	32.4	0.8052	28.4	59.0	27.4	0.7444	23.9	33.5	15.8	0.5890	42.1
MLP	Q	60.8	27.6	0.7526	25.8	77.6	33.0	0.8528	18.7	59.8	26.7	0.7558	21.3	60.0	27.6	0.7481	25.1	32.2	18.8	0.5756	41.7
	C	59.4	18.3	0.7477	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7494	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5897	41.7
	Q+C	62.6	30.1	0.7665	27.1	78.3	35.7	0.8652	18.4	66.9	32.0	0.7921	28.2	58.5	26.2	0.7427	24.0	34.9	21.8	0.5913	43.7
DNN (BERT)	Q	59.3	35.6	0.7461	26.9	73.4	40.8	0.8371	18.6	61.5	36.0	0.7601	25.7	58.7	33.9	0.7411	25.5	33.3	27.4	0.5788	43.5
	C	59.4	18.3	0.7493	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7592	21.3	58.8	18.5	0.7398	25.1	34.8	12.9	0.5909	41.7
	Q+C	61.4	34.6	0.7591	27.4	74.7	38.1	0.8452	18.8	65.2	35.9	0.7825	27.9	61.8	35.7	0.7600	25.7	35.3	27.1	0.5903	43.4
TwoTower (BERT)	Q	60.8	31.2	0.7541	26.3	74.5	34.2	0.8431	18.1	62.9	33.3	0.7676	24.8	60.9	31.9	0.7504	25.3	35.0	24.0	0.5898	42.6
	C	59.4	18.3	0.7496	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7595	21.3	58.8	18.5	0.7403	25.1	34.8	12.9	0.5916	41.7
	Q+C	61.7	31.2	0.7604	26.7	75.0	33.6	0.8458	18.3	63.8	34.3	0.7720	27.5	62.0	32.4	0.7607	24.7	36.9	23.9	0.6011	42.3
Adaptive-RAG (BERT)	Q	61.6	30.9	0.7587	26.4	75.1	33.9	0.8469	18.0	63.6	34.0	0.7702	26.1	60.6	31.5	0.7520	25.2	36.7	23.3	0.5985	42.4
	C	59.4	18.3	0.7497	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7597	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5916	41.7
	Q+C	62.7	30.8	0.7673	27.2	75.4	32.3	0.8510	18.7	67.8	34.8	0.8007	28.8	61.5	34.1	0.7565	25.0	37.6	23.2	0.6047	42.7
MBA-RAG (BERT)	Q	60.7	31.1	0.7501	26.2	74.4	34.7	0.8387	18.0	63.3	32.8	0.7669	24.8	60.0	30.5	0.7450	24.6	35.0	24.1	0.5849	42.7
	C	59.4	18.3	0.7462	25.8	73.2	21.1	0.8334	18.7	60.2	18.8	0.7593	21.3	58.8	18.5	0.7394	25.1	34.8	12.9	0.5870	41.7
	Q+C	62.9	30.5	0.7647	27.1	75.8	31.4	0.8476	18.7	66.9	33.8	0.7899	27.7	62.4	35.1	0.7578	25.5	37.7	23.8	0.6058	42.8
RouterDC (BERT)	Q	60.0	33.6	0.7488	26.5	73.2	37.2	0.8341	18.7	63.5	36.1	0.7673	24.9	61.3	34.4	0.7556	25.2	33.3	25.2	0.5804	42.3
	C	59.4	18.3	0.7474	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7595	21.3	58.8	18.5	0.7372	25.1	34.8	12.9	0.5851	41.7
	Q+C	61.3	34.5	0.7593	27.2	73.8	37.5	0.8404	18.9	65.6	38.7	0.7865	28.0	61.4	34.8	0.7568	25.3	36.5	26.4	0.5997	42.7
DNN (BGE)	Q	58.0	35.2	0.7378	27.6	71.4	38.4	0.8238	18.9	58.7	33.6	0.7433	26.2	57.9	34.7	0.7386	25.9	33.8	30.8	0.5810	45.1
	C	59.4	18.3	0.7495	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7592	21.3	58.8	18.5	0.7403	25.1	34.8	12.9	0.5916	41.7
	Q+C	58.3	34.6	0.7424	27.4	72.1	37.5	0.8305	18.7	60.4	35.4	0.7561	27.6	58.4	35.3	0.7398	25.1	32.5	28.5	0.5793	44.3
TwoTower (BGE)	Q	59.4	29.5	0.7459	26.4	72.7	30.2	0.8313	18.2	61.2	31.4	0.7565	24.2	60.0	30.9	0.7461	25.3	34.4	26.0	0.5875	43.3
	C	59.4	18.3	0.7496	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7592	21.3	58.8	18.5	0.7408	25.1	34.8	12.9	0.5916	41.7
	Q+C	59.9	29.9	0.7497	27.1	73.2	30.7	0.8339	18.6	62.8	35.1	0.7729	26.2	61.3	32.5	0.7554	25.1	33.6	23.4	0.5812	43.9
Adaptive-RAG (BGE)	Q	60.2	31.2	0.7500	26.4	73.3	32.6	0.8330	18.3	61.8	31.9	0.7591	24.9	60.0	30.9	0.7492	24.9	36.1	28.3	0.5973	42.9
	C	59.4	18.3	0.7493	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7590	21.3	58.8	18.5	0.7393	25.1	34.8	12.9	0.5916	41.7
	Q+C	60.2	30.4	0.7518	26.9	73.6	31.3	0.8368	18.7	62.5	31.4	0.7665	26.1	61.1	34.2	0.7569	25.1	34.7	25.6	0.5879	43.1
MBA-RAG (BGE)	Q	56.4	31.8	0.7190	26.8	71.2	35.0	0.8133	18.0	58.7	32.9	0.7371	26.1	56.2	32.3	0.7195	25.6	28.7	25.1	0.5401	43.9
	C	59.4	18.3	0.7471	25.8	73.2	21.1	0.8328	18.7	60.2	18.8	0.7587	21.3	58.8	18.5	0.7412	25.1	34.8	12.9	0.5907	41.7
	Q+C	59.2	29.2	0.7390	27.1	72.8	29.7	0.8265	18.5	62.5	31.8	0.7629	27.2	58.5	32.3	0.7327	25.7	33.5	24.3	0.5733	43.4
RouterDC (BGE)	Q	57.7	33.7	0.7359	27.3	72.0	37.0	0.8279	18.4	57.8	32.4	0.7358	25.2	58.9	33.4	0.7424	25.7	31.6	29.0	0.5689	45.7
	C	59.4	18.3	0.7488	25.8	73.2	21.1	0.8382	18.7	60.2	18.8	0.7596	21.3	58.8	18.5	0.7389	25.1	34.8	12.9	0.5892	41.7
	Q+C	60.0	33.8	0.7502	27.1	74.0	38.2	0.8397	18.9	60.5	32.9	0.7524	26.6	59.5	35.0	0.7442	25.6	35.3	25.8	0.5938	42.8
Oracle	–	100.0	100.0	1.0000	40.6	100.0	–	–	26.8	100.0	–	–	39.7	100.0	–	–	41.2	100.0	–	–	65.1

bilities, computed as the average of $1/\text{rank}$ of the ground-truth paradigm across all queries. **Correct Rate (C.R)** measures the proportion of queries for which the paradigm selected by the router is judged as correct by LLM-as-a-Judge, where queries predicted as "cannot answer" fall back to the globally most frequent optimal paradigm. Table 3 in the main text reports R.A, macro F1 and C.R; MRR appears in the full results of Appendix D.4.

D.4 Complete Results

Table 24 presents the comprehensive router evaluation across all architectures, feature configurations (query-only, corpus-only, and Q+C), and datasets, including the logistic-regression and SVM routers omitted from Table 3 for space. Because the traditional ML routers observe only twelve distinct input vectors (three query types $\times$ four corpora), they converge to closely matching per-cell mappings, which is why their results differ only marginally from one another. Results are reported for both BERT and BGE encoders; BGE results serve as a robustness check confirming that the Q+C advantage holds across different pretrained representations.

D.5 The *Cannot Answer* Class

A natural question is whether the *cannot answer* class inflates routing metrics by allowing a router to skip difficult queries. It does not, for two reasons.

Abstention is scored, not skipped. As defined in Section 3.5, a query predicted as *cannot answer* is neither counted as correct nor as incorrect. It falls back to the most frequent non-abstaining paradigm of the training fold, and its correctness is read off that paradigm's actual generated answer. Abstention therefore changes which paradigm is scored, not whether the outcome counts.

Predicted abstention tracks genuine unanswerability. Under the ground-truth labels, the fraction of queries for which no paradigm is correct varies sharply by corpus on DeepSeek-V3: 48.4% on MuSiQue, 40.1% on QuALITY, 49.5% on Legal, and 15.4% on Medical (39.2% overall). Router-predicted abstention rates follow the same ordering, peaking on MuSiQue and Legal and collapsing on Medical, which indicates that the predicted rate reflects the difficulty profile of each corpus rather than indiscriminate deferral. Routers with richer query encodings track this profile most closely, while simpler classifiers over-abstain relative to it.

Removing the class does not change the conclusions. We additionally re-train every router without the *cannot answer* class, relabeling each such training instance to its highest-Semantic-F1 paradigm. Across the full router–backbone grid the change in correct rate stays within -0.5 to $+1.0$ percentage points. The accompanying cost shift is determined entirely by which paradigm the five-class router had used as its fallback, and carries no accuracy implication. The class is thus best understood as an abstention signal with downstream deployment value, such as confidence-aware execution or deferral to a human, rather than as the mechanism behind the routing gains reported in Section 4.1.

E Reproducibility and Extensibility

Reproducibility. Decoding temperatures are fixed per stage, namely $T = 0.0$ for knowledge-graph triplet extraction and for LLM-as-a-Judge evaluation, $T = 0.7$ for query generation and for the retrieval-free LLM-only baseline, $T = 0.3$ for answer generation, and $T = 0.1$ for the IterativeRAG evaluator (Tables 6, 13 and 14). Retrieval, query augmentation, and cross-validation splits are seeded, and dependency versions are pinned, so that a run reproduces the reported numbers. Every tunable parameter is declared in a central configuration layer rather than inline, and the values used throughout this paper are listed in Appendix B.1.

Extensibility. The framework separates four concerns, so that each of the following extensions is a localized change rather than a pipeline rewrite. (i) *A new corpus* passes through the same preparation pipeline—chunking, embedding, corpus-indicator computation, and verify-then-filter query validation—after which every existing paradigm runs on it unchanged. (ii) *A new RAG paradigm* implements the shared indexing, retrieval, and generation interface; once it returns answers and token counts in the common format, evaluation, cost decomposition, and label construction are reused as-is. (iii) *A new router* consumes the existing query features, corpus indicators, and paradigm labels, since feature loading, model definition, and evaluation are factored separately. (iv) *A new judge* replaces one configuration entry, as the evaluation prompt and the three-level label schema are independent of the judge model.